Transformer_Captioning

May 23, 2026

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

Student Declaration (must be completed)

SUNet id:

Zunmi

Did you use generative AI tools (e.g., ChatGPT, Copilot, etc.)?

- [ ] No
- [1] Yes (please describe below)

If yes, describe how you used generative AI and for what parts of the assignment :

guide

Confirm that all submitted work reflects your own understanding and that you did not rely on AI to complete substantial portions of the assignment:

- [1] I confirm

```
In [5]: # Server setup cell.
        import os
        import sys

        ASSIGNMENT_DIR = "/data/users/huanghao/a3/assignment3"
        assert os.path.isdir(ASSIGNMENT_DIR), f"[!] Assignment directory not found: {ASSIGNMEN

        if ASSIGNMENT_DIR not in sys.path:
            sys.path.append(ASSIGNMENT_DIR)

        # Download the COCO dataset if it doesn't already exist.
        datasets_dir = os.path.join(ASSIGNMENT_DIR, "cs231n", "datasets")
        os.chdir(datasets_dir)
        !bash get_coco_dataset.sh
        os.chdir(ASSIGNMENT_DIR)
        print(f"Working directory: {os.getcwd()}")

Working directory: /data/users/huanghao/a3/assignment3
```

1 Image Captioning with Transformers

You have now implemented a vanilla RNN and for the task of image captioning. In this notebook you will implement key pieces of a transformer decoder to accomplish the same task.

NOTE: This notebook will be primarily written in PyTorch rather than NumPy, unlike the RNN notebook.

In [6]:
```
# Setup cell.
import time, os, json
import numpy as np
import matplotlib.pyplot as plt
import sys
import types
import importlib

from cs231n.gradient_check import eval_numerical_gradient, eval_numerical_gradient_arra
from cs231n.transformer_layers import *
from cs231n.captioning_solver_transformer import CaptioningSolverTransformer
from cs231n.classifiers.transformer import CaptioningTransformer
from cs231n.coco_utils import load_coco_data, sample_coco_minibatch, decode_captions
from cs231n.image_utils import image_from_url

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # Set default size of plots.
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

if "imp" not in sys.modules:
    imp = types.ModuleType("imp")
    imp.reload = importlib.reload
    sys.modules["imp"] = imp

%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """ returns relative error """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

2 COCO Dataset

As in the previous notebooks, we will use the COCO dataset for captioning.

In [7]:
```
# Load COCO data from disk into a dictionary.
data = load_coco_data(pca_features=True)
```

```
    # Print out all the keys and values from the data dictionary.
    for k, v in data.items():
        if type(v) == np.ndarray:
            print(k, type(v), v.shape, v.dtype)
        else:
            print(k, type(v), len(v))
```

```
base dir  /data/users/huanghao/a3/assignment3/cs231n/datasets/coco_captioning
train_captions <class 'numpy.ndarray'> (400135, 17) int32
train_image_idxs <class 'numpy.ndarray'> (400135,) int32
val_captions <class 'numpy.ndarray'> (195954, 17) int32
val_image_idxs <class 'numpy.ndarray'> (195954,) int32
train_features <class 'numpy.ndarray'> (82783, 512) float32
val_features <class 'numpy.ndarray'> (40504, 512) float32
idx_to_word <class 'list'> 1004
word_to_idx <class 'dict'> 1004
train_urls <class 'numpy.ndarray'> (82783,) <U63
val_urls <class 'numpy.ndarray'> (40504,) <U63
```

3 Transformer

As you have seen, RNNs are incredibly powerful but often slow to train. Further, RNNs struggle to encode long-range dependencies (though LSTMs are one way of mitigating the issue). In 2017, Vaswani et al introduced the Transformer in their paper "Attention Is All You Need" to a) introduce parallelism and b) allow models to learn long-range dependencies. The paper not only led to famous models like BERT and GPT in the natural language processing community, but also an explosion of interest across fields, including vision. While here we introduce the model in the context of image captioning, the idea of attention itself is much more general.

4 Transformer: Multi-Headed Attention

4.0.1 Dot-Product Attention

Recall that attention can be viewed as an operation on a query $q \in \mathbb{R}^d$, a set of value vectors $\{v_1, \ldots, v_n\}, v_i \in \mathbb{R}^d$, and a set of key vectors $\{k_1, \ldots, k_n\}, k_i \in \mathbb{R}^d$, specified as

$$c = \sum_{i=1}^{n} v_i \alpha_i \tag{1}$$

$$\alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^{n} \exp(k_j^\top q)} \tag{2}$$

$$\tag{3}$$

where α_i are frequently called the "attention weights", and the output $c \in \mathbb{R}^d$ is a correspondingly weighted average over the value vectors.

4.0.2 Self-Attention

In Transformers, we perform self-attention, which means that the values, keys and query are derived from the input $X \in \mathbb{R}^{\ell \times d}$, where ℓ is our sequence length. Specifically, we learn parameter matrices $V, K, Q \in \mathbb{R}^{d \times d}$ to map our input X as follows:

$$v_i = Vx_i \;\; i \in \{1, \ldots, \ell\} \tag{4}$$
$$k_i = Kx_i \;\; i \in \{1, \ldots, \ell\} \tag{5}$$
$$q_i = Qx_i \;\; i \in \{1, \ldots, \ell\} \tag{6}$$

4.0.3 Multi-Headed Scaled Dot-Product Attention

In the case of multi-headed attention, we learn a parameter matrix for each head, which gives the model more expressivity to attend to different parts of the input. Let h be number of heads, and Y_i be the attention output of head i. Thus we learn individual matrices Q_i, K_i and V_i. To keep our overall computation the same as the single-headed case, we choose $Q_i \in \mathbb{R}^{d \times d/h}$, $K_i \in \mathbb{R}^{d \times d/h}$ and $V_i \in \mathbb{R}^{d \times d/h}$. Adding in a scaling term $\frac{1}{\sqrt{d/h}}$ to our simple dot-product attention above, we have

$$Y_i = \text{softmax}\left(\frac{(XQ_i)(XK_i)^\top}{\sqrt{d/h}}\right)(XV_i) \tag{7}$$

where $Y_i \in \mathbb{R}^{\ell \times d/h}$, where ℓ is our sequence length.

In our implementation, we apply dropout to the attention weights (though in practice it could be used at any step):

$$Y_i = \text{dropout}\left(\text{softmax}\left(\frac{(XQ_i)(XK_i)^\top}{\sqrt{d/h}}\right)\right)(XV_i) \tag{8}$$

Finally, then the output of the self-attention is a linear transformation of the concatenation of the heads:

$$Y = [Y_1; \ldots; Y_h]A \tag{9}$$

were $A \in \mathbb{R}^{d \times d}$ and $[Y_1; \ldots; Y_h] \in \mathbb{R}^{\ell \times d}$.

Implement multi-headed scaled dot-product attention in the `MultiHeadAttention` class in the file `cs231n/transformer_layers.py`. The code below will check your implementation. The relative error should be less than `7e-3`.

In [8]:
```
torch.manual_seed(231)

# Choose dimensions such that they are all unique for easier debugging:
# Specifically, the following values correspond to N=1, H=2, T=3, E//H=4, and E=8.
batch_size = 1
sequence_length = 3
embed_dim = 8
attn = MultiHeadAttention(embed_dim, num_heads=2)
attn.eval()

# Self-attention.
```

```
        data = torch.randn(batch_size, sequence_length, embed_dim)
        self_attn_output = attn(query=data, key=data, value=data)

        # Masked self-attention.
        mask = torch.randn(sequence_length, sequence_length) < 0.5
        masked_self_attn_output = attn(query=data, key=data, value=data, attn_mask=mask)

        # Attention using two inputs.
        other_data = torch.randn(batch_size, sequence_length, embed_dim)
        attn_output = attn(query=data, key=other_data, value=other_data)

        expected_self_attn_output = np.asarray([[
            [-0.1030,  0.2090,  0.7481, -0.4116, -0.2772,  0.2357, -0.2740, -0.2798],
            [-0.1794,  0.1637,  0.6930, -0.3435, -0.2300,  0.2457, -0.2504, -0.2323],
            [-0.1861,  0.1567,  0.6677, -0.3078, -0.2731,  0.2302, -0.2714, -0.2082]]])

        expected_masked_self_attn_output = np.asarray([[
            [-0.1209,  0.1806,  0.8056, -0.4654, -0.2262,  0.2552, -0.2544, -0.2812],
            [-0.2374,  0.1183,  0.7423, -0.3777, -0.1460,  0.2716, -0.2213, -0.2077],
            [-0.0851,  0.2423,  0.7941, -0.3778, -0.4924,  0.1512, -0.4241, -0.1740]]])

        expected_attn_output = np.asarray([[
            [-0.1965,  0.0702,  0.5227, -0.1388, -0.3530,  0.2163, -0.2643, -0.1902],
            [-0.3592, -0.0899,  0.5836,  0.0029, -0.3643,  0.2667, -0.1973, -0.1698],
            [-0.3795,  0.0433,  0.5634, -0.0749, -0.4379,  0.2714, -0.2207, -0.1980]]])

        print('self_attn_output error: ', rel_error(expected_self_attn_output, self_attn_output
        print('masked_self_attn_output error: ', rel_error(expected_masked_self_attn_output, ma
        print('attn_output error: ', rel_error(expected_attn_output, attn_output.detach().numpy
```

```
self_attn_output error:  0.00017949659048965922
masked_self_attn_output error:  9.809519326315612e-05
attn_output error:  0.006226898176165351
```

5 Positional Encoding

While transformers are able to easily attend to any part of their input, the attention mechanism has no concept of token order. However, for many tasks (especially natural language processing), relative token order is very important. To recover this, the authors add a positional encoding to the embeddings of individual word tokens.

Let us define a matrix $P \in \mathbb{R}^{l \times d}$, where $P_{ij} = $

$$\begin{cases} \sin\left(i \cdot 10000^{-\frac{j}{d}}\right) & \text{if j is even} \\ \cos\left(i \cdot 10000^{-\frac{(j-1)}{d}}\right) & \text{otherwise} \end{cases}$$

Rather than directly passing an input $X \in \mathbb{R}^{l \times d}$ to our network, we instead pass $X + P$.

Implement this layer in `PositionalEncoding` in `cs231n/transformer_layers.py`. Once you are done, run the following to perform a simple test of your implementation. You should see errors on the order of `e-3` or less.

```
In [9]: torch.manual_seed(231)

        batch_size = 1
        sequence_length = 2
        embed_dim = 6
        data = torch.randn(batch_size, sequence_length, embed_dim)

        pos_encoder = PositionalEncoding(embed_dim)
        pos_encoder.eval()
        output = pos_encoder(data)

        expected_pe_output = np.asarray([[[-1.1106,  1.0014,  1.5280, -0.0778, -0.6964,  1.145
                                           [ 0.8125, -0.4303,  0.4981,  0.7320,  1.1380,  1.533
        print('pe_output error: ', rel_error(expected_pe_output, output.detach().numpy()))

pe_output error:  0.00021699471301977143
```

6 Inline Question 1

Several key design decisions were made in designing the scaled dot product attention we introduced above. Explain why the following choices were beneficial: 1. Using multiple attention heads as opposed to one. 2. Dividing by $\sqrt{d/h}$ before applying the softmax function. Recall that d is the feature dimension and h is the number of heads. 3. Adding a linear transformation to the output of the attention operation.

Only one or two sentences per choice is necessary, but be sure to be specific in addressing what would have happened without each given implementation detail, why such a situation would be suboptimal, and how the proposed implementation improves the situation.

Your Answer: 1. Multiple attention heads: Using multiple heads allows the model to attend to different types of relationships or different positions in parallel, since each head learns its own query/key/value projections. With only one head, all attention information would be forced through a single similarity pattern, which can limit the model's ability to capture diverse dependencies. 2. Dividing by $\sqrt{d/h}$: The dot products QK^T grow larger in magnitude as the per-head dimension d/h increases, which can make the softmax distribution extremely peaked. If the softmax saturates, gradients become small and training becomes unstable; dividing by $\sqrt{d/h}$ keeps the scores at a reasonable scale. 3. Linear transformation after attention: After concatenating the heads, the output is just a collection of separate head-specific features. A final linear transformation mixes information across heads and projects the combined representation back into the model's feature space, allowing the model to learn useful interactions between different attention heads.

7 Transformer Decoder Block

Transformer decoder layer consists of three modules: (1) self attention to process input sequence of vectors, (2) cross attention to process based on available context (i.e. image features in our case), (3) feedforward module to process each vector of the sequence independently. Complete the implementation of `TransformerDecoderLayer` in `cs231n/transformer_layers.py` and test it below. The relative error should be less than 1e-6.

The Transformer decoder layer has three main components: (1) a self-attention module that processes the input sequence of vectors, (2) a cross-attention module that incorporates additional context (e.g., image features in our case), and (3) a feedforward module that independently processes each vector in the sequence. Complete the implementation of `TransformerDecoderLayer` in `cs231n/transformer_layers.py` and test it below. The relative error should be less than 1e-6.

```
In [10]: torch.manual_seed(231)
         np.random.seed(231)

         N, T, TM, D = 1, 4, 5, 12

         decoder_layer = TransformerDecoderLayer(D, 2, 4*D)
         decoder_layer.eval()
         tgt = torch.randn(N, T, D)
         memory = torch.randn(N, TM, D)
         tgt_mask = torch.randn(T, T) < 0.5

         output = decoder_layer(tgt, memory, tgt_mask)

         expected_output = np.asarray([
             [[ 0.27005595, -0.21056601,  0.63376445, -0.35391954,  0.60071295,
               -1.7566615,  -0.1314159,  -1.7659538,  -0.5575517,   0.680143,
                1.8622686,   0.7291234],
              [-0.6080135,   0.4421802,  -0.07590749, -0.12093157,  0.94722354,
               -1.0223433,   0.47332686, -1.3958666,   2.1890075,  -1.0380646,
                0.9437816,  -0.7343922],
              [-0.89690155,  0.60447216,  0.10688882,  0.03492759,  1.8388084,
               -0.9772293,   0.7621334,  -0.668234,    1.3260399,  -1.8417019,
                0.21718435, -0.506388],
              [-0.70961505, -0.02918372, -0.39894593, -0.8069395,   0.04967685,
                0.29345992,  0.7871747,  -1.3172938,   1.4744806,   1.2633642,
                1.1611108,  -1.767289]]
         ])
         print('error: ', rel_error(expected_output, output.detach().numpy()))
```

```
error:  8.386729410631422e-08
```

8 Transformer for Image Captioning

Now that you have implemented the previous layers, you can combine them to build a Transformer-based image captioning model. Open the file `cs231n/classifiers/transformer.py` and look at the `CaptioningTransformer` class.

Implement the `forward` function of the class. After doing so, run the following to check your forward pass using a small test case; you should see error on the order of `e-5` or less.

```
In [11]: torch.manual_seed(231)
         np.random.seed(231)

         N, D, W = 4, 20, 30
         word_to_idx = {'<NULL>': 0, 'cat': 2, 'dog': 3}
         V = len(word_to_idx)
         T = 3

         transformer = CaptioningTransformer(
             word_to_idx,
             input_dim=D,
             wordvec_dim=W,
             num_heads=2,
             num_layers=2,
             max_length=30
         )
         transformer.eval()

         features = torch.randn(N, D)
         captions = torch.randint(0, V, (N, T))

         scores = transformer(features, captions)

         expected_scores = np.asarray([
             [[ 0.62165695, -0.6203744,  -1.116114  ],
              [ 0.6641097,  -0.60309553, -1.3479083 ],
              [ 0.62117404, -0.79205745, -1.1095243 ]],
             [[ 0.62160987, -0.6214724,  -1.1171186 ],
              [ 0.6093992,  -0.61166143, -1.3365762 ],
              [ 0.62108713, -0.79322916, -1.1104958 ]],
             [[ 0.6406587,  -0.57419956, -1.1300566 ],
              [ 0.6075228,  -0.60872054, -1.3372457 ],
              [ 0.6471472,  -0.8270494,  -1.1064028 ]],
             [[ 0.64029807, -0.5755552,  -1.1284486 ],
              [ 0.60708034, -0.6099963,  -1.3356377 ],
              [ 0.59367496, -0.8346836,  -1.0914496 ]]
         ])

         print('scores error: ', rel_error(expected_scores, scores.detach().numpy()))
```

```
scores error:  1.4597957790730836e-07
```

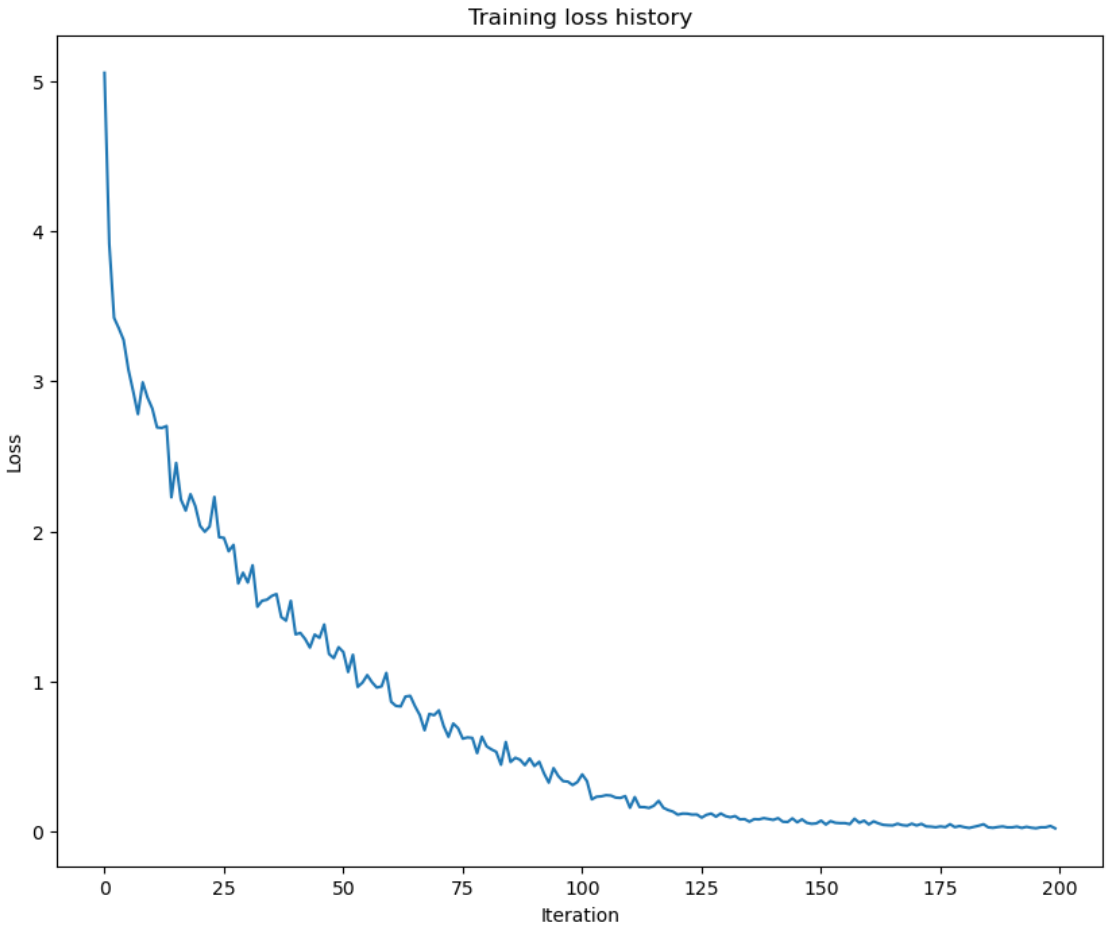
9 Overfit Transformer Captioning Model on Small Data

Run the following to overfit the Transformer-based captioning model on the same small dataset as we used for the RNN previously.

In [12]:
```
torch.manual_seed(231)
np.random.seed(231)

data = load_coco_data(max_train=50)

transformer = CaptioningTransformer(
          word_to_idx=data['word_to_idx'],
          input_dim=data['train_features'].shape[1],
          wordvec_dim=256,
          num_heads=2,
          num_layers=2,
          max_length=30
        )


transformer_solver = CaptioningSolverTransformer(transformer, data, idx_to_word=data[
           num_epochs=100,
           batch_size=25,
           learning_rate=0.001,
           verbose=True, print_every=10,
         )

transformer_solver.train()

# Plot the training losses.
plt.plot(transformer_solver.loss_history)
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('Training loss history')
plt.show()
```

```
base dir  /data/users/huanghao/a3/assignment3/cs231n/datasets/coco_captioning
(Iteration 1 / 200) loss: 5.055427
(Iteration 11 / 200) loss: 2.819050
(Iteration 21 / 200) loss: 2.036877
(Iteration 31 / 200) loss: 1.659048
(Iteration 41 / 200) loss: 1.313489
(Iteration 51 / 200) loss: 1.195412
(Iteration 61 / 200) loss: 0.864514
(Iteration 71 / 200) loss: 0.806814
(Iteration 81 / 200) loss: 0.566810
```

```
(Iteration 91 / 200) loss: 0.435672
(Iteration 101 / 200) loss: 0.380263
(Iteration 111 / 200) loss: 0.157928
(Iteration 121 / 200) loss: 0.111748
(Iteration 131 / 200) loss: 0.102176
(Iteration 141 / 200) loss: 0.077055
(Iteration 151 / 200) loss: 0.072140
(Iteration 161 / 200) loss: 0.046706
(Iteration 171 / 200) loss: 0.039755
(Iteration 181 / 200) loss: 0.028434
(Iteration 191 / 200) loss: 0.027322
```



Print final training loss. You should see a final loss of less than 0.05 .

```
In [13]: print('Final loss: ', transformer_solver.loss_history[-1])

Final loss:  0.020686615
```

10 Transformer Sampling at Test Time

The sampling code has been written for you. You can simply run the following to compare with the previous results with the RNN. As before the training results should be much better than the validation set results, given how little data we trained on.

In [14]:
```
# If you get an error, the URL just no longer exists, so don't worry!
# You can re-sample as many times as you want.
for split in ['train', 'val']:
    minibatch = sample_coco_minibatch(data, split=split, batch_size=2)
    gt_captions, features, urls = minibatch
    gt_captions = decode_captions(gt_captions, data['idx_to_word'])

    sample_captions = transformer.sample(features, max_length=30)
    sample_captions = decode_captions(sample_captions, data['idx_to_word'])

    for gt_caption, sample_caption, url in zip(gt_captions, sample_captions, urls):
        img = image_from_url(url)
        # Skip missing URLs.
        if img is None: continue
        plt.imshow(img)
        plt.title('%s\n%s\nGT:%s' % (split, sample_caption, gt_caption))
        plt.axis('off')
        plt.show()
```

```
URL Error:  Gone http://farm1.staticflickr.com/202/487987371_489a65d670_z.jpg
```

train
a <UNK> decorated living room with a big tv in it <END>
GT:<START> a <UNK> decorated living room with a big tv in it <END>

URL Error: Not Found http://farm1.staticflickr.com/25/44101107_9491d72776_z.jpg

11 Vision Transformer (ViT)

Dosovitskiy et. al. showed that applying a transformer model on a sequence of image patches (referred to as Vision Transformer) not only achieves impressive performance but also scales more effectively than convolutional neural networks when trained on large datasets. We will build a version of Vision Transformer using our existing implementation of transformer components and train it on the CIFAR-10 dataset.

Vision Transformer converts input image into a sequence of patches of fixed size and embed each patch into a latent vector. In `cs231n/transformer_layers.py`, complete the implementation of `PatchEmbedding` and test it below. You should see relative error less than 1e-4.

```
In [15]: from cs231n.transformer_layers import PatchEmbedding

         torch.manual_seed(231)
         np.random.seed(231)

         N = 2
         HW = 16
```

```
PS = 8
D = 8

patch_embedding = PatchEmbedding(
    img_size=HW,
    patch_size=PS,
    embed_dim=D
)

x = torch.randn(N, 3, HW, HW)
output = patch_embedding(x)

expected_output = np.asarray([
        [[-0.6312704 ,  0.02531429,  0.6112642 , -0.49089882,
          0.01412961, -0.6959372 , -0.32862484, -0.45402682],
         [ 0.18816411, -0.08142513, -0.9829535 , -0.23975623,
          -0.23109074,  0.97950286, -0.40997326,  0.7457837 ],
         [ 0.01810865,  0.15780598, -0.91804236,  0.36185235,
          0.8379501 ,  1.0191797 , -0.29667392,  0.20322265],
         [-0.18697818, -0.45137224, -0.40339014, -1.4381214 ,
          -0.43450755,  0.7651071 , -0.83683825, -0.16360264]],

        [[-0.39786366,  0.16201034, -0.19008337, -1.0602452 ,
          -0.28693503,  0.09791763,  0.26614824,  0.41781986],
         [ 0.35146567, -0.4469593 , -0.1841726 ,  0.45757473,
          -0.61304873, -0.29104248, -0.16124889, -0.14987172],
         [-0.2996967 ,  0.27353522, -0.09929767,  0.01973832,
          -1.2312065 , -0.6374332 , -0.22963578,  0.55696607],
         [-0.93818814,  0.02465284, -0.21117875,  1.1860403 ,
          -0.06137538, -0.21062079, -0.094347  ,  0.50032747]]])

print('error: ', rel_error(expected_output, output.detach().numpy()))
```

```
error:  9.627833207220966e-07
```

The sequence of patch vectors is processed by transformer encoder layers, each consisting of a self-attention and a feed-forward module. Since all vectors attend to one another, attention masking is not strictly necessary. However, we still implement it for the sake of consistency.

Implement `TransformerEncoderLayer` in `cs231n/transformer_layers.py` and test it below. You should see relative error less than 1e-6.

```
In [16]: torch.manual_seed(231)
         np.random.seed(231)

         from cs231n.transformer_layers import TransformerEncoderLayer

         N, T, TM, D = 1, 4, 5, 12
```

```
        encoder_layer = TransformerEncoderLayer(D, 2, 4*D)
        encoder_layer.eval()
        x = torch.randn(N, T, D)
        x_mask = torch.randn(T, T) < 0.5

        output = encoder_layer(x, x_mask)

        expected_output = np.asarray([
            [[-0.48887438, -0.16470742,  0.7249629,  -1.0907345,   1.5764195,
               0.17484017,  0.8643941,  -0.5209561,  -1.6651545,  -1.1943513,
               1.4809005,   0.30326116],
             [-0.14663944,  1.052812,   -0.96227,    -0.20350897, -2.1478589,
               1.2637341,   0.04015125, -0.945255,    1.408903,   -0.43051198,
               0.37110543,  0.69933903],
             [ 0.08718029,  0.91050833, -1.208469,    1.1440394,  -1.9794976,
              -0.38342738, -0.47407156, -0.17345548,  0.11738186,  1.9534125,
               0.3525986,  -0.3461999],
             [-0.81389004,  0.76213264, -0.7094313,  -1.5332246,   1.3837544,
               0.4091982,   0.6569406,   0.7305365,  -1.197077,   -1.3698943,
               0.58292854,  1.0980265]]
        ])

        print('error: ', rel_error(expected_output, output.detach().numpy()))
```

```
error:  4.162545799227941e-08
```

Take a look at the `VisionTransformer` implementation in `cs231n/classifiers/transformer.py`.

For classification, ViT divides the input image into patches and processes the sequence of patch vectors using a transformer. Finally, all the patch vectors are average-pooled and used to predict the image class. We will use the same 1D sinusoidal positional encoding to inject ordering information, though 2D sinusoidal and learned positional encodings are also valid choices.

Complete the ViT forward pass and test it below. You should see relative error less than 1e-6.

```
In [17]: torch.manual_seed(231)
         np.random.seed(231)
         from cs231n.classifiers.transformer import VisionTransformer

         imgs = torch.randn(3, 3, 32, 32)
         transformer = VisionTransformer()
         transformer.eval()

         scores = transformer(imgs)

         expected_scores = np.asarray(
             [[-0.15802915,  0.15734261, -0.06002094, -0.1860142,  -0.09376919,
```

```
                -0.08837911,  0.16224845,  0.0168601,   0.20606893,  0.13079852],
               [-0.12838936,  0.20798579, -0.0620573,  -0.11705838, -0.14088857,
                -0.11387511,  0.18839046,  0.03078492,  0.17386372,  0.06414902],
               [-0.09435399,  0.20495278, -0.03362986, -0.12232997, -0.15535004,
                -0.10380758,  0.21114832,  0.03481449,  0.18798208,  0.12231653]])

        print('scores error: ', rel_error(expected_scores, scores.detach().numpy()))
```

```
scores error:  2.25376991479709e-07
```

We will first verify our implementation by overfitting it on one training batch. Tune learning rate and weight decay accordingly.

```
In [18]: from torchvision import transforms
         from torchvision.datasets import CIFAR10
         from tqdm.auto import tqdm
         from torch.utils.data import DataLoader

         train_data = CIFAR10(root='data', train=True, transform=transforms.ToTensor(), downloa
         test_data = CIFAR10(root='data', train=False, transform=transforms.ToTensor(), downloa
```

```
Downloading https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz to data/cifar-10-python.ta


100%|| 170498071/170498071 [26:19<00:00, 107969.84it/s]


Extracting data/cifar-10-python.tar.gz to data
Files already downloaded and verified
```

```
In [22]: learning_rate = 1e-3  # Experiment with this
         weight_decay = 1.e-4  # Experiment with this


         batch = next(iter(DataLoader(train_data, batch_size=64, shuffle=False)))
         model = VisionTransformer(dropout=0.0)
         loss_criterion = torch.nn.CrossEntropyLoss()
         optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate, weight_decay=weight
         model.train()

         epochs = 100
         for epoch in range(epochs):
             imgs, target = batch
             out = model(imgs)
             loss = loss_criterion(out, target)

             optimizer.zero_grad()
```

```
            loss.backward()
            optimizer.step()

            top1 = (out.argmax(-1) == target).float().mean().item()
            if epoch % 10 == 0:
              print(f"[{epoch}/{epochs}] Loss {loss.item():.6f}, Top-1 Accuracy: {top1:.3f}")
```

```
[0/100] Loss 2.313155, Top-1 Accuracy: 0.109
[10/100] Loss 2.106844, Top-1 Accuracy: 0.188
[20/100] Loss 1.846448, Top-1 Accuracy: 0.312
[30/100] Loss 1.686087, Top-1 Accuracy: 0.359
[40/100] Loss 1.350152, Top-1 Accuracy: 0.547
[50/100] Loss 1.084778, Top-1 Accuracy: 0.656
[60/100] Loss 0.673579, Top-1 Accuracy: 0.781
[70/100] Loss 0.383649, Top-1 Accuracy: 0.891
[80/100] Loss 0.096298, Top-1 Accuracy: 1.000
[90/100] Loss 0.032766, Top-1 Accuracy: 1.000
```

```
In [20]: # You should get perfect 1.00 accuracy
         print(f"Overfitting ViT on one batch. Top-1 accuracy: {top1}")
```

```
Overfitting ViT on one batch. Top-1 accuracy: 0.3125
```

Now we will train it on the entire dataset.

```
In [26]: from cs231n.classification_solver_vit import ClassificationSolverViT

         ################################################################################
         # TODO: Train a Vision Transformer model that achieves over 0.45 test          #
         # accuracy on CIFAR-10 after 2 epochs by adjusting the model architecture      #
         # and/or training parameters as needed.                                        #
         #                                                                              #
         # Note: If you want to use a GPU runtime, go to `Runtime > Change runtime      #
         # type` and set `Hardware accelerator` to `GPU`. This will reset Colab,        #
         # so make sure to rerun the entire notebook from the beginning afterward.      #
         ################################################################################


         learning_rate = 1e-3
         weight_decay = 1.e-4
         batch_size = 64
         model = VisionTransformer()  # You may want to change the default params.




         ################################################################################
```

```
#                                 END OF YOUR CODE                             #
################################################################################

solver = ClassificationSolverViT(
    train_data=train_data,
    test_data=test_data,
    model=model,
    num_epochs = 2,  # Don't change this
    learning_rate = learning_rate,
    weight_decay = weight_decay,
    batch_size = batch_size,
)

solver.train('cuda' if torch.cuda.is_available() else 'cpu')
```

In [27]:
```
print(f"Accuracy on test set: {solver.results['best_test_acc']}")
```

```
Accuracy on test set: 0.4703
```

12 Inline Question 2

Despite their recent success in large-scale image recognition tasks, ViTs often lag behind traditional CNNs when trained on smaller datasets. What underlying factor contribute to this performance gap? What techniques can be used to improve the performance of ViTs on small datasets?

Your Answer: ViTs often perform worse than CNNs on small datasets because they have weaker built-in image inductive biases. CNNs naturally encode locality, translation equivariance, and hierarchical feature extraction through convolution and pooling, while ViTs must learn many of these visual structures from data, so they usually need larger datasets or stronger pretraining.

To improve ViT performance on small datasets, we can use large-scale pretraining followed by fine-tuning, stronger data augmentation,modifications that add CNN-like biases, such as convolutional patch embeddings, hybrid CNN-ViT backbones, or hierarchical/local-window attention.

13 Inline Question 3

How does the computational cost of the self-attention layers in a ViT change if we independently make the following changes? Please ignore the computation cost of QKV and output projection.

(i) Double the hidden dimension.
(ii) Double the height and width of the input image.
(iii) Double the patch size.
(iv) Double the number of layers.

Your Answer: $(i)\ 2\times, \quad (ii)\ 16\times, \quad (iii)\ \frac{1}{16}\times, \quad (iv)\ 2\times$

In [ ]:

Self_Supervised_Learning

May 23, 2026

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

In [16]:
```
# Server setup cell.
import os
import sys

ASSIGNMENT_DIR = "/data/users/huanghao/a3/assignment3"
assert os.path.isdir(ASSIGNMENT_DIR), f"[!] Assignment directory not found: {ASSIGNME

if ASSIGNMENT_DIR not in sys.path:
    sys.path.append(ASSIGNMENT_DIR)

# Download the assignment datasets if they do not already exist.
datasets_dir = os.path.join(ASSIGNMENT_DIR, "cs231n", "datasets")
os.chdir(datasets_dir)
!bash get_datasets.sh
os.chdir(ASSIGNMENT_DIR)
print(f"Working directory: {os.getcwd()}")
```

```
Working directory: /data/users/huanghao/a3/assignment3
```

0.1 Using GPU

If the server has a GPU available, the notebook will use it automatically through `torch.cuda.is_available()`. You can rerun the setup cell after changing environments if needed.

1 Self-Supervised Learning

1.1 What is self-supervised learning?

Modern day machine learning requires lots of labeled data. But often times it's challenging and/or expensive to obtain large amounts of human-labeled data. Is there a way we could ask machines to automatically learn a model which can generate good visual representations without a labeled dataset? Yes, enter self-supervised learning!

Self-supervised learning (SSL) allows models to automatically learn a "good" representation space using the data in a given dataset without the need for their labels. Specifically, if our dataset were a bunch of images, then self-supervised learning allows a model to learn and generate a "good" representation vector for images.

The reason SSL methods have seen a surge in popularity is because the learnt model continues to perform well on other datasets as well i.e. new datasets on which the model was not trained on!

1.2 What makes a "good" representation?

A "good" representation vector needs to capture the important features of the image as it relates to the rest of the dataset. This means that images in the dataset representing semantically similar entities should have similar representation vectors, and different images in the dataset should have different representation vectors. For example, two images of an apple should have similar representation vectors, while an image of an apple and an image of a banana should have different representation vectors.

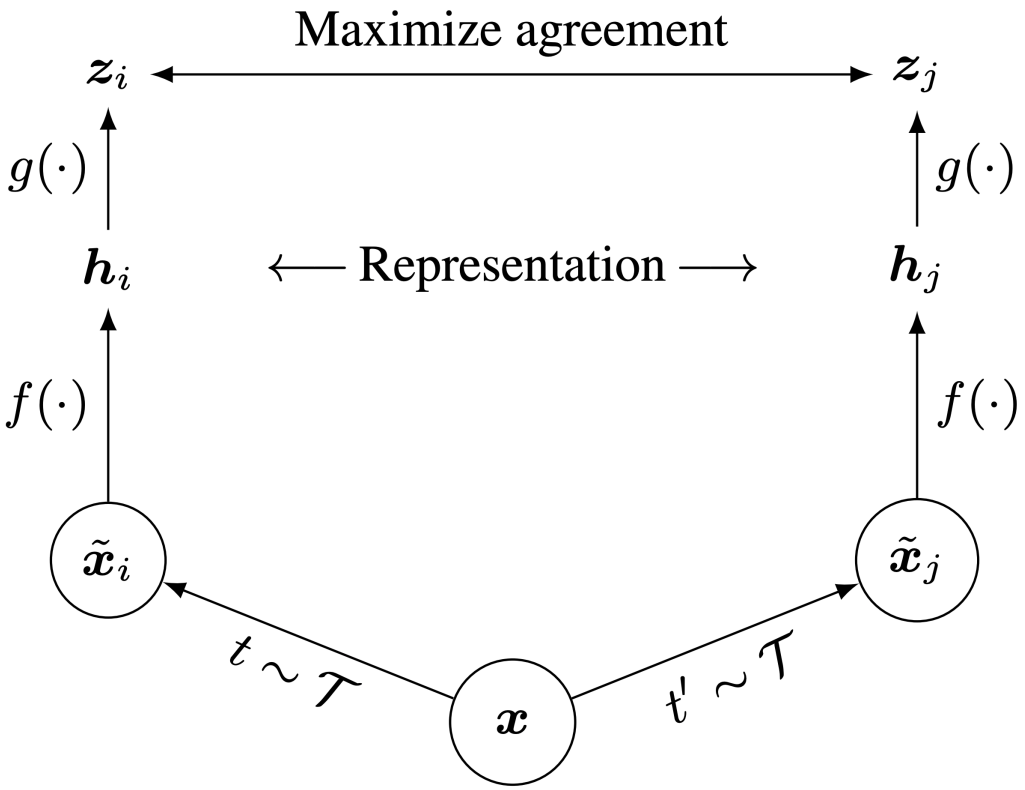
1.3 Contrastive Learning: SimCLR

Recently, SimCLR introduces a new architecture which uses **contrastive learning** to learn good visual representations. Contrastive learning aims to learn similar representations for similar images and different representations for different images. As we will see in this notebook, this simple idea allows us to train a surprisingly good model without using any labels.

Specifically, for each image in the dataset, SimCLR generates two differently augmented views of that image, called a **positive pair**. Then, the model is encouraged to generate similar representation vectors for this pair of images. See below for an illustration of the architecture (Figure 2 from the paper).

In [17]:
```
# Run this cell to view the SimCLR architecture.
from IPython.display import Image
Image('images/simclr_fig2.png', width=500)
```

Out[17]:



Given an image **x**, SimCLR uses two different data augmentation schemes **t** and **t′** to generate the positive pair of images $\tilde{x}_i$ and $\tilde{x}_j$. f is a basic encoder net that extracts representation vectors from the augmented data samples, which yields h_i and h_j, respectively. Finally, a small neural network projection head g maps the representation vectors to the space where the contrastive loss is applied. The goal of the contrastive loss is to maximize agreement between the final vectors $z_i = g(h_i)$ and $z_j = g(h_j)$. We will discuss the contrastive loss in more detail later, and you will get to implement it.

After training is completed, we throw away the projection head g and only use f and the representation h to perform downstream tasks, such as classification. You will get a chance to finetune a layer on top of a trained SimCLR model for a classification task and compare its performance with a baseline model (without self-supervised learning).

1.4 Pretrained Weights

For your convenience, we have given you pretrained weights (trained for ~18 hours on CIFAR-10) for the SimCLR model. Run the following cell to download pretrained model weights to be used later. (This will take ~1 minute)

```
In [18]: %%bash
         DIR=pretrained_model/
         if [ ! -d "$DIR" ]; then
```

```
    mkdir "$DIR"
fi

URL=http://downloads.cs.stanford.edu/downloads/cs231n/pretrained_simclr_model.pth
# Try this if above doesn't work.
# URL=http://cs231n.stanford.edu/2025/storage/a3/pretrained_simclr_model.pth
FILE=pretrained_model/pretrained_simclr_model.pth
if [ ! -f "$FILE" ]; then
    echo "Downloading weights..."
    wget "$URL" -O "$FILE"
fi
```

In [19]:
```
# Setup cell.
%pip install thop
import torch
import os
import sys
import types
import importlib
import pandas as pd
import numpy as np
import torch.optim as optim
import torch.nn as nn
import random
from thop import profile, clever_format
from torch.utils.data import DataLoader
from torchvision.datasets import CIFAR10
import matplotlib.pyplot as plt
%matplotlib inline

if "imp" not in sys.modules:
    imp = types.ModuleType("imp")
    imp.reload = importlib.reload
    sys.modules["imp"] = imp

%load_ext autoreload
%autoreload 2

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
Requirement already satisfied: thop in /data/users/huanghao/miniconda3/envs/cs224n-a3/lib/pytho
Requirement already satisfied: torch in /data/users/huanghao/miniconda3/envs/cs224n-a3/lib/pyth
Requirement already satisfied: filelock in /data/users/huanghao/miniconda3/envs/cs224n-a3/lib/p
Requirement already satisfied: typing-extensions in /data/users/huanghao/miniconda3/envs/cs224n
Requirement already satisfied: sympy in /data/users/huanghao/miniconda3/envs/cs224n-a3/lib/pyth
Requirement already satisfied: networkx in /data/users/huanghao/miniconda3/envs/cs224n-a3/lib/p
Requirement already satisfied: jinja2 in /data/users/huanghao/miniconda3/envs/cs224n-a3/lib/pyt
Requirement already satisfied: nvidia-cuda-nvrtc-cu11==11.7.99 in /data/users/huanghao/miniconda
```

```
Requirement already satisfied: nvidia-cuda-runtime-cu11==11.7.99 in /data/users/huanghao/minico
Requirement already satisfied: nvidia-cuda-cupti-cu11==11.7.101 in /data/users/huanghao/minicon
Requirement already satisfied: nvidia-cudnn-cu11==8.5.0.96 in /data/users/huanghao/miniconda3/e
Requirement already satisfied: nvidia-cublas-cu11==11.10.3.66 in /data/users/huanghao/miniconda
Requirement already satisfied: nvidia-cufft-cu11==10.9.0.58 in /data/users/huanghao/miniconda3/
Requirement already satisfied: nvidia-curand-cu11==10.2.10.91 in /data/users/huanghao/miniconda
Requirement already satisfied: nvidia-cusolver-cu11==11.4.0.1 in /data/users/huanghao/miniconda
Requirement already satisfied: nvidia-cusparse-cu11==11.7.4.91 in /data/users/huanghao/minicond
Requirement already satisfied: nvidia-nccl-cu11==2.14.3 in /data/users/huanghao/miniconda3/envs
Requirement already satisfied: nvidia-nvtx-cu11==11.7.91 in /data/users/huanghao/miniconda3/env
Requirement already satisfied: triton==2.0.0 in /data/users/huanghao/miniconda3/envs/cs224n-a3/
Requirement already satisfied: setuptools in /data/users/huanghao/miniconda3/envs/cs224n-a3/lib
Requirement already satisfied: wheel in /data/users/huanghao/miniconda3/envs/cs224n-a3/lib/pyth
Requirement already satisfied: cmake in /data/users/huanghao/miniconda3/envs/cs224n-a3/lib/pyth
Requirement already satisfied: lit in /data/users/huanghao/miniconda3/envs/cs224n-a3/lib/python
Requirement already satisfied: MarkupSafe>=0.23 in /data/users/huanghao/miniconda3/envs/cs224n-
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /data/users/huanghao/miniconda3/envs/cs224

[notice] A new release of pip is available: 23.3.2 -> 25.0.1
[notice] To update, run: pip install --upgrade pip
Note: you may need to restart the kernel to use updated packages.
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

2 Data Augmentation

Our first step is to perform data augmentation. Implement the `compute_train_transform()` function in `cs231n/simclr/data_utils.py` to apply the following random transformations:

1. Randomly resize and crop to 32x32.
2. Horizontally flip the image with probability 0.5
3. With a probability of 0.8, apply color jitter (see `compute_train_transform()` for definition)
4. With a probability of 0.2, convert the image to grayscale

Now complete `compute_train_transform()` and `CIFAR10Pair.__getitem__()` in `cs231n/simclr/data_utils.py` to apply the data augmentation transform and generate $\tilde{x}_i$ and $\tilde{x}_j$.

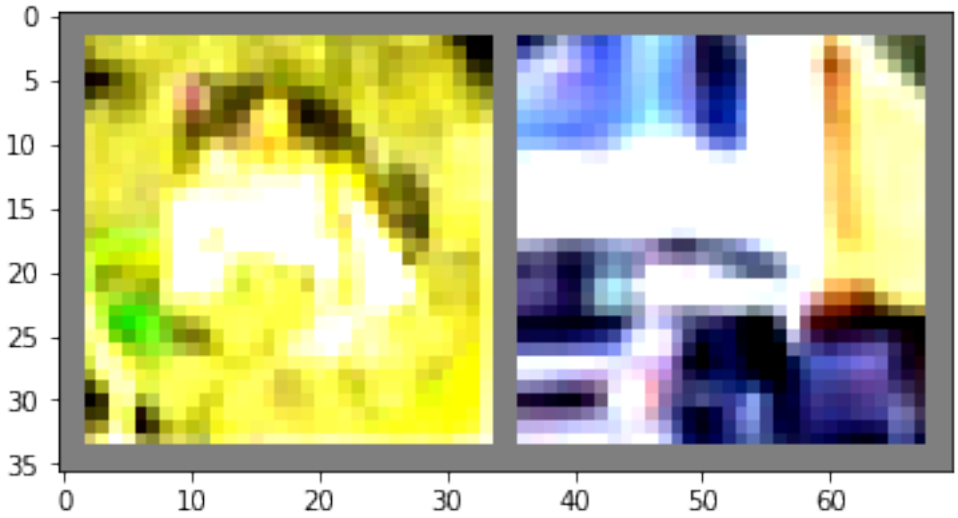
Test to make sure that your data augmentation code is correct:

```
In [20]: from cs231n.simclr.data_utils import *
         from cs231n.simclr.contrastive_loss import *

         answers = torch.load('simclr_sanity_check.key')

In [21]: from PIL import Image
         import torchvision
         from torchvision.datasets import CIFAR10
```

```
def test_data_augmentation(correct_output=None):
    train_transform = compute_train_transform(seed=2147483647)
    trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True,
    trainloader = torch.utils.data.DataLoader(trainset, batch_size=2, shuffle=False, r
    dataiter = iter(trainloader)
    images, labels = next(dataiter)
    img = torchvision.utils.make_grid(images)
    img = img / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()
    output = images

    print("Maximum error in data augmentation: %g"%rel_error( output.numpy(), correct_

# Should be less than 1e-07.
test_data_augmentation(answers['data_augmentation'])
```

```
Files already downloaded and verified
```

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255]
```




```
Maximum error in data augmentation: 1
```

3 Base Encoder and Projection Head

The next steps are to apply the base encoder and projection head to the augmented samples $\tilde{x}_i$ and $\tilde{x}_j$.

The base encoder f extracts representation vectors for the augmented samples. The SimCLR paper found that using deeper and wider models improved performance and thus chose ResNet to use as the base encoder. The output of the base encoder are the representation vectors $h_i = f(\tilde{x}_i)$ and $h_j = f(\tilde{x}_j)$.

The projection head g is a small neural network that maps the representation vectors h_i and h_j to the space where the contrastive loss is applied. The paper found that using a nonlinear projection head improved the representation quality of the layer before it. Specifically, they used a MLP with one hidden layer as the projection head g. The contrastive loss is then computed based on the outputs $z_i = g(h_i)$ and $z_j = g(h_j)$.

We provide implementations of these two parts in `cs231n/simclr/model.py`. Please skim through the file and make sure you understand the implementation.

4 SimCLR: Contrastive Loss

A mini-batch of N training images yields a total of $2N$ data-augmented examples. For each positive pair (i, j) of augmented examples, the contrastive loss function aims to maximize the agreement of vectors z_i and z_j. Specifically, the loss is the normalized temperature-scaled cross entropy loss and aims to maximize the agreement of z_i and z_j relative to all other augmented examples in the batch:

$$l\ (i,j) = -\log \frac{\exp(\ \text{sim}(z_i, z_j)\ /\ \tau)}{\sum_{k=1}^{2N} \mathbb{1}_{k \neq i} \exp(\ \text{sim}(z_i, z_k)\ /\ \tau)}$$

where $\mathbb{1} \in \{0, 1\}$ is an indicator function that outputs 1 if $k \neq i$ and 0 otherwise. τ is a temperature parameter that determines how fast the exponentials increase.

$\text{sim}(z_i, z_j) = \frac{z_i \cdot z_j}{||z_i|| ||z_j||}$ is the (normalized) dot product between vectors z_i and z_j. The higher the similarity between z_i and z_j, the larger the dot product is, and the larger the numerator becomes. The denominator normalizes the value by summing across z_i and all other augmented examples k in the batch. The range of the normalized value is $(0, 1)$, where a high score close to 1 corresponds to a high similarity between the positive pair (i, j) and low similarity between i and other augmented examples k in the batch. The negative log then maps the range $(0, 1)$ to the loss values $(\text{inf}, 0)$.

The total loss is computed across all positive pairs (i, j) in the batch. Let $z = [z_1, z_2, ..., z_{2N}]$ include all the augmented examples in the batch, where $z_1 ... z_N$ are outputs of the left branch, and $z_{N+1} ... z_{2N}$ are outputs of the right branch. Thus, the positive pairs are (z_k, z_{k+N}) for $\forall k \in [1, N]$.

Then, the total loss L is:

$$L = \frac{1}{2N} \sum_{k=1}^{N} [\ l(k,\ k+N) + l(k+N,\ k)\ ]$$

NOTE: this equation is slightly different from the one in the paper. We've rearranged the ordering of the positive pairs in the batch, so the indices are different. The rearrangement makes it easier to implement the code in vectorized form.

We'll walk through the steps of implementing the loss function in vectorized form. Implement the functions `sim`, `simclr_loss_naive` in `cs231n/simclr/contrastive_loss.py`. Test your code by running the sanity checks below.

```
In [22]: from cs231n.simclr.contrastive_loss import *
         answers = torch.load('simclr_sanity_check.key')
```

```
In [23]: def test_sim(left_vec, right_vec, correct_output):
             output = sim(left_vec, right_vec).cpu().numpy()
             print("Maximum error in sim: %g"%rel_error(correct_output.numpy(), output))

         # Should be less than 1e-07.
         test_sim(answers['left'][0], answers['right'][0], answers['sim'][0])
         test_sim(answers['left'][1], answers['right'][1], answers['sim'][1])
```

```
Maximum error in sim: 3.81097e-08
Maximum error in sim: 0
```

```
In [24]: def test_loss_naive(left, right, tau, correct_output):
             naive_loss = simclr_loss_naive(left, right, tau).item()
             print("Maximum error in simclr_loss_naive: %g"%rel_error(correct_output, naive_los

         # Should be less than 1e-07.
         test_loss_naive(answers['left'], answers['right'], 5.0, answers['loss']['5.0'])
         test_loss_naive(answers['left'], answers['right'], 1.0, answers['loss']['1.0'])
```

```
Maximum error in simclr_loss_naive: 0
Maximum error in simclr_loss_naive: 5.65617e-08
```

Now implement the vectorized version by implementing `sim_positive_pairs`, `compute_sim_matrix`, `simclr_loss_vectorized` in `cs231n/simclr/contrastive_loss.py`. Test your code by running the sanity checks below.

```
In [25]: def test_sim_positive_pairs(left, right, correct_output):
             sim_pair = sim_positive_pairs(left, right).cpu().numpy()
             print("Maximum error in sim_positive_pairs: %g"%rel_error(correct_output.numpy(),

         # Should be less than 1e-07.
         test_sim_positive_pairs(answers['left'], answers['right'], answers['sim'])
```

```
Maximum error in sim_positive_pairs: 3.81097e-08
```

```
In [26]: def test_sim_matrix(left, right, correct_output):
             out = torch.cat([left, right], dim=0)
             sim_matrix = compute_sim_matrix(out).cpu()
             assert torch.isclose(sim_matrix, correct_output).all(), "correct: {}. got: {}".fo
             print("Test passed!")

         test_sim_matrix(answers['left'], answers['right'], answers['sim_matrix'])
```

```
Test passed!
```

```
In [27]: def test_loss_vectorized(left, right, tau, correct_output):
             vec_loss = simclr_loss_vectorized(left, right, tau, device=left.device).item()
             print("Maximum error in loss_vectorized: %g"%rel_error(correct_output, vec_loss))

         # Should be less than 1e-07.
         test_loss_vectorized(answers['left'], answers['right'], 5.0, answers['loss']['5.0'])
         test_loss_vectorized(answers['left'], answers['right'], 1.0, answers['loss']['1.0'])
```

```
Maximum error in loss_vectorized: 0
Maximum error in loss_vectorized: 5.65617e-08
```

5 Implement the train function

Complete the `train()` function in `cs231n/simclr/utils.py` to obtain the model's output and use `simclr_loss_vectorized` to compute the loss. (Please take a look at the `Model` class in `cs231n/simclr/model.py` to understand the model pipeline and the returned values)

```
In [28]: from cs231n.simclr.data_utils import *
         from cs231n.simclr.model import *
         from cs231n.simclr.utils import *
```

5.0.1 Train the SimCLR model

Run the following cells to load in the pretrained weights and continue to train a little bit more. This part will take ~10 minutes and will output to `pretrained_model/trained_simclr_model.pth`.

NOTE: Don't worry about logs such as *'[WARN] Cannot find rule for ...'*. These are related to another module used in the notebook. You can verify the integrity of your code changes through our provided prompts and comments.

```
In [29]: # Do not modify this cell.
         feature_dim = 128
         temperature = 0.5
         k = 200
         batch_size = 64
         epochs = 1
         temperature = 0.5
         percentage = 0.5
         pretrained_path = './pretrained_model/pretrained_simclr_model.pth'

         # Prepare the data.
         train_transform = compute_train_transform()
         train_data = CIFAR10Pair(root='data', train=True, transform=train_transform, download=
         train_data = torch.utils.data.Subset(train_data, list(np.arange(int(len(train_data)*pe
         train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True, num_workers
```

```
        test_transform = compute_test_transform()
        memory_data = CIFAR10Pair(root='data', train=True, transform=test_transform, download=
        memory_loader = DataLoader(memory_data, batch_size=batch_size, shuffle=False, num_work
        test_data = CIFAR10Pair(root='data', train=False, transform=test_transform, download=
        test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=False, num_workers=

        # Set up the model and optimizer config.
        model = Model(feature_dim)
        model.load_state_dict(torch.load(pretrained_path, map_location='cpu'), strict=False)
        model = model.to(device)
        flops, params = profile(model, inputs=(torch.randn(1, 3, 32, 32).to(device),))
        flops, params = clever_format([flops, params])
        print('# Model Params: {} FLOPs: {}'.format(params, flops))
        optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-6)
        c = len(memory_data.classes)

        # Training loop.
        results = {'train_loss': [], 'test_acc@1': [], 'test_acc@5': []} #<< -- output

        if not os.path.exists('results'):
            os.mkdir('results')
        best_acc = 0.0
        for epoch in range(1, epochs + 1):
            train_loss = train(model, train_loader, optimizer, epoch, epochs, batch_size=batch
            results['train_loss'].append(train_loss)
            test_acc_1, test_acc_5 = test(model, memory_loader, test_loader, epoch, epochs, c
            results['test_acc@1'].append(test_acc_1)
            results['test_acc@5'].append(test_acc_5)

            # Save statistics.
            if test_acc_1 > best_acc:
                best_acc = test_acc_1
                torch.save(model.state_dict(), './pretrained_model/trained_simclr_model.pth')
```

```
Files already downloaded and verified
Files already downloaded and verified
Files already downloaded and verified
[INFO] Register count_convNd() for <class 'torch.nn.modules.conv.Conv2d'>.
[INFO] Register count_normalization() for <class 'torch.nn.modules.batchnorm.BatchNorm2d'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.activation.ReLU'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.container.Sequential'>.
[INFO] Register count_adap_avgpool() for <class 'torch.nn.modules.pooling.AdaptiveAvgPool2d'>.
[INFO] Register count_linear() for <class 'torch.nn.modules.linear.Linear'>.
[INFO] Register count_normalization() for <class 'torch.nn.modules.batchnorm.BatchNorm1d'>.
# Model Params: 24.62M FLOPs: 1.31G


Train Epoch: [1/1] Loss: 3.2580: 100%|| 390/390 [00:36<00:00, 10.82it/s]
Feature extracting: 100%|| 782/782 [00:08<00:00, 97.14it/s]
```

```
Test Epoch: [1/1] Acc@1:83.63% Acc@5:99.37%: 100%|| 157/157 [00:03<00:00, 48.95it/s]
```

6 Finetune a Linear Layer for Classification!

Now it's time to put the representation vectors to the test!

We remove the projection head from the SimCLR model and slap on a linear layer to finetune for a simple classification task. All layers before the linear layer are frozen, and only the weights in the final linear layer are trained. We compare the performance of the SimCLR + finetuning model against a baseline model, where no self-supervised learning is done beforehand, and all weights in the model are trained. You will get to see for yourself the power of self-supervised learning and how the learned representation vectors improve downstream task performance.

6.1 Baseline: Without Self-Supervised Learning

First, let's take a look at the baseline model. We'll remove the projection head from the SimCLR model and slap on a linear layer to finetune for a simple classification task. No self-supervised learning is done beforehand, and all weights in the model are trained. Run the following cells.

NOTE: Don't worry if you see low but reasonable performance.

In [30]:
```
class Classifier(nn.Module):
    def __init__(self, num_class):
        super(Classifier, self).__init__()

        # Encoder.
        self.f = Model().f

        # Classifier.
        self.fc = nn.Linear(2048, num_class, bias=True)

    def forward(self, x):
        x = self.f(x)
        feature = torch.flatten(x, start_dim=1)
        out = self.fc(feature)
        return out
```

In [31]:
```
# Do not modify this cell.
feature_dim = 128
temperature = 0.5
k = 200
batch_size = 128
epochs = 10
percentage = 0.1

train_transform = compute_train_transform()
train_data = CIFAR10(root='data', train=True, transform=train_transform, download=True
trainset = torch.utils.data.Subset(train_data, list(np.arange(int(len(train_data)*perc
train_loader = DataLoader(trainset, batch_size=batch_size, shuffle=True, num_workers=
```

```
        test_transform = compute_test_transform()
        test_data = CIFAR10(root='data', train=False, transform=test_transform, download=True)
        test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=False, num_workers=

        model = Classifier(num_class=len(train_data.classes)).to(device)
        for param in model.f.parameters():
            param.requires_grad = False

        flops, params = profile(model, inputs=(torch.randn(1, 3, 32, 32).to(device),))
        flops, params = clever_format([flops, params])
        print('# Model Params: {} FLOPs: {}'.format(params, flops))
        optimizer = optim.Adam(model.fc.parameters(), lr=1e-3, weight_decay=1e-6)
        no_pretrain_results = {'train_loss': [], 'train_acc@1': [], 'train_acc@5': [],
                               'test_loss': [], 'test_acc@1': [], 'test_acc@5': []}

        best_acc = 0.0
        for epoch in range(1, epochs + 1):
            train_loss, train_acc_1, train_acc_5 = train_val(model, train_loader, optimizer, e
            no_pretrain_results['train_loss'].append(train_loss)
            no_pretrain_results['train_acc@1'].append(train_acc_1)
            no_pretrain_results['train_acc@5'].append(train_acc_5)
            test_loss, test_acc_1, test_acc_5 = train_val(model, test_loader, None, epoch, epo
            no_pretrain_results['test_loss'].append(test_loss)
            no_pretrain_results['test_acc@1'].append(test_acc_1)
            no_pretrain_results['test_acc@5'].append(test_acc_5)
            if test_acc_1 > best_acc:
                best_acc = test_acc_1

        # Print the best test accuracy.
        print('Best top-1 accuracy without self-supervised learning: ', best_acc)
```

```
Files already downloaded and verified
Files already downloaded and verified
[INFO] Register count_convNd() for <class 'torch.nn.modules.conv.Conv2d'>.
[INFO] Register count_normalization() for <class 'torch.nn.modules.batchnorm.BatchNorm2d'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.activation.ReLU'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.container.Sequential'>.
[INFO] Register count_adap_avgpool() for <class 'torch.nn.modules.pooling.AdaptiveAvgPool2d'>.
[INFO] Register count_linear() for <class 'torch.nn.modules.linear.Linear'>.
# Model Params: 23.52M FLOPs: 1.31G


Train Epoch: [1/10] Loss: 2.5538 ACC@1: 10.44% ACC@5: 50.70%: 100%|| 40/40 [00:02<00:00, 18.58i
Test Epoch: [1/10] Loss: 2.3129 ACC@1: 11.81% ACC@5: 52.43%: 100%|| 79/79 [00:02<00:00, 30.98it
Train Epoch: [2/10] Loss: 2.4306 ACC@1: 10.86% ACC@5: 51.68%: 100%|| 40/40 [00:01<00:00, 20.06i
Test Epoch: [2/10] Loss: 2.6969 ACC@1: 10.09% ACC@5: 55.31%: 100%|| 79/79 [00:02<00:00, 32.16it
Train Epoch: [3/10] Loss: 2.3920 ACC@1: 11.74% ACC@5: 53.20%: 100%|| 40/40 [00:01<00:00, 21.32i
Test Epoch: [3/10] Loss: 2.5002 ACC@1: 10.41% ACC@5: 53.08%: 100%|| 79/79 [00:02<00:00, 32.33it
```

```
Train Epoch: [4/10] Loss: 2.4011 ACC@1: 11.70% ACC@5: 53.96%: 100%|| 40/40 [00:02<00:00, 18.54i
Test Epoch: [4/10] Loss: 2.5989 ACC@1: 10.36% ACC@5: 52.54%: 100%|| 79/79 [00:02<00:00, 29.74it
Train Epoch: [5/10] Loss: 2.4138 ACC@1: 12.54% ACC@5: 54.62%: 100%|| 40/40 [00:02<00:00, 19.61i
Test Epoch: [5/10] Loss: 2.7139 ACC@1: 14.97% ACC@5: 54.22%: 100%|| 79/79 [00:02<00:00, 28.85it
Train Epoch: [6/10] Loss: 2.3960 ACC@1: 12.42% ACC@5: 53.98%: 100%|| 40/40 [00:02<00:00, 19.62i
Test Epoch: [6/10] Loss: 2.3873 ACC@1: 13.37% ACC@5: 54.47%: 100%|| 79/79 [00:02<00:00, 29.14it
Train Epoch: [7/10] Loss: 2.3648 ACC@1: 13.24% ACC@5: 54.26%: 100%|| 40/40 [00:02<00:00, 19.47i
Test Epoch: [7/10] Loss: 2.4513 ACC@1: 11.96% ACC@5: 55.73%: 100%|| 79/79 [00:02<00:00, 29.63it
Train Epoch: [8/10] Loss: 2.3830 ACC@1: 11.96% ACC@5: 55.18%: 100%|| 40/40 [00:02<00:00, 19.77i
Test Epoch: [8/10] Loss: 2.4713 ACC@1: 14.09% ACC@5: 59.37%: 100%|| 79/79 [00:02<00:00, 29.62it
Train Epoch: [9/10] Loss: 2.3807 ACC@1: 13.14% ACC@5: 56.82%: 100%|| 40/40 [00:02<00:00, 19.45i
Test Epoch: [9/10] Loss: 2.6677 ACC@1: 10.08% ACC@5: 57.11%: 100%|| 79/79 [00:02<00:00, 30.50it
Train Epoch: [10/10] Loss: 2.3999 ACC@1: 12.80% ACC@5: 57.38%: 100%|| 40/40 [00:01<00:00, 20.99
Test Epoch: [10/10] Loss: 2.4388 ACC@1: 15.38% ACC@5: 57.90%: 100%|| 79/79 [00:02<00:00, 32.95i

Best top-1 accuracy without self-supervised learning:  15.379999999999999
```

6.2 With Self-Supervised Learning

Let's see how much improvement we get with self-supervised learning. Here, we pretrain the SimCLR model using the simclr loss you wrote, remove the projection head from the SimCLR model, and use a linear layer to finetune for a simple classification task.

In [32]:
```python
# Do not modify this cell.
feature_dim = 128
temperature = 0.5
k = 200
batch_size = 128
epochs = 10
percentage = 0.1
pretrained_path = './pretrained_model/trained_simclr_model.pth'

train_transform = compute_train_transform()
train_data = CIFAR10(root='data', train=True, transform=train_transform, download=True
trainset = torch.utils.data.Subset(train_data, list(np.arange(int(len(train_data)*perc
train_loader = DataLoader(trainset, batch_size=batch_size, shuffle=True, num_workers=
test_transform = compute_test_transform()
test_data = CIFAR10(root='data', train=False, transform=test_transform, download=True)
test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=False, num_workers=

model = Classifier(num_class=len(train_data.classes))
model.load_state_dict(torch.load(pretrained_path, map_location='cpu'), strict=False)
model = model.to(device)
for param in model.f.parameters():
    param.requires_grad = False
```

```
        flops, params = profile(model, inputs=(torch.randn(1, 3, 32, 32).to(device),))
        flops, params = clever_format([flops, params])
        print('# Model Params: {} FLOPs: {}'.format(params, flops))
        optimizer = optim.Adam(model.fc.parameters(), lr=1e-3, weight_decay=1e-6)
        pretrain_results = {'train_loss': [], 'train_acc@1': [], 'train_acc@5': [],
                            'test_loss': [], 'test_acc@1': [], 'test_acc@5': []}

        best_acc = 0.0
        for epoch in range(1, epochs + 1):
            train_loss, train_acc_1, train_acc_5 = train_val(model, train_loader, optimizer, e
            pretrain_results['train_loss'].append(train_loss)
            pretrain_results['train_acc@1'].append(train_acc_1)
            pretrain_results['train_acc@5'].append(train_acc_5)
            test_loss, test_acc_1, test_acc_5 = train_val(model, test_loader, None, epoch, epo
            pretrain_results['test_loss'].append(test_loss)
            pretrain_results['test_acc@1'].append(test_acc_1)
            pretrain_results['test_acc@5'].append(test_acc_5)
            if test_acc_1 > best_acc:
                best_acc = test_acc_1

        # Print the best test accuracy. You should see a best top-1 accuracy of >=70%.
        print('Best top-1 accuracy with self-supervised learning: ', best_acc)
```

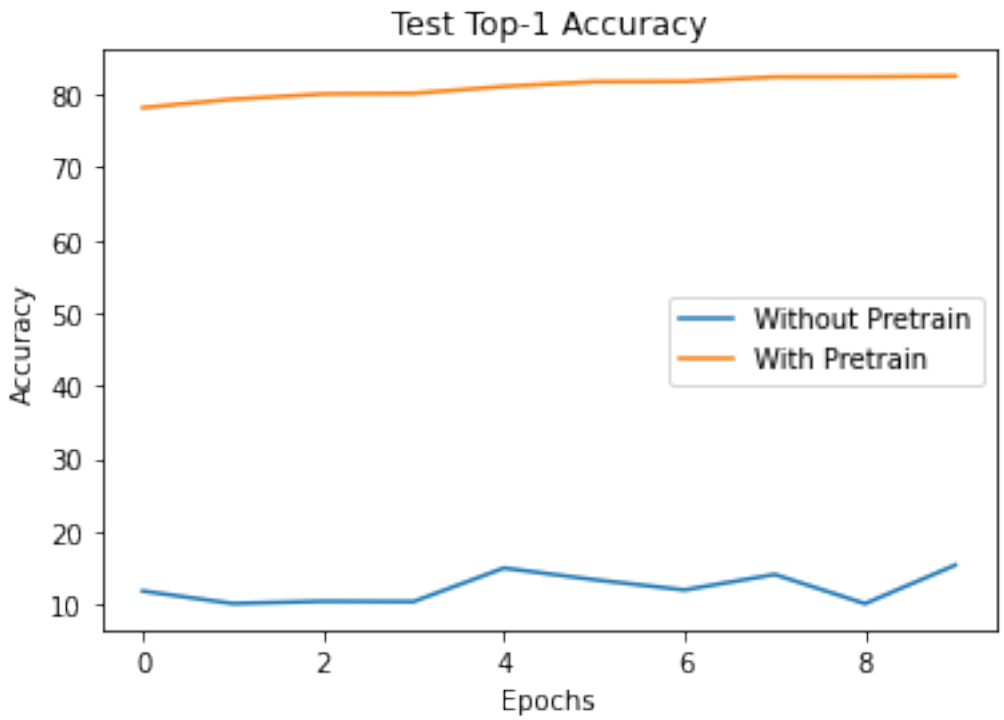
```
Files already downloaded and verified
Files already downloaded and verified
[INFO] Register count_convNd() for <class 'torch.nn.modules.conv.Conv2d'>.
[INFO] Register count_normalization() for <class 'torch.nn.modules.batchnorm.BatchNorm2d'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.activation.ReLU'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.container.Sequential'>.
[INFO] Register count_adap_avgpool() for <class 'torch.nn.modules.pooling.AdaptiveAvgPool2d'>.
[INFO] Register count_linear() for <class 'torch.nn.modules.linear.Linear'>.
# Model Params: 23.52M FLOPs: 1.31G


Train Epoch: [1/10] Loss: 1.8251 ACC@1: 64.34% ACC@5: 93.64%: 100%|| 40/40 [00:01<00:00, 22.00i
Test Epoch: [1/10] Loss: 1.3319 ACC@1: 78.18% ACC@5: 98.26%: 100%|| 79/79 [00:02<00:00, 32.53it
Train Epoch: [2/10] Loss: 1.1920 ACC@1: 76.02% ACC@5: 97.58%: 100%|| 40/40 [00:01<00:00, 21.46i
Test Epoch: [2/10] Loss: 0.9372 ACC@1: 79.34% ACC@5: 98.35%: 100%|| 79/79 [00:02<00:00, 32.18it
Train Epoch: [3/10] Loss: 0.9350 ACC@1: 76.50% ACC@5: 97.80%: 100%|| 40/40 [00:01<00:00, 20.90i
Test Epoch: [3/10] Loss: 0.7739 ACC@1: 80.06% ACC@5: 98.78%: 100%|| 79/79 [00:02<00:00, 29.05it
Train Epoch: [4/10] Loss: 0.8413 ACC@1: 77.08% ACC@5: 97.70%: 100%|| 40/40 [00:02<00:00, 18.89i
Test Epoch: [4/10] Loss: 0.7021 ACC@1: 80.15% ACC@5: 98.65%: 100%|| 79/79 [00:02<00:00, 29.39it
Train Epoch: [5/10] Loss: 0.7698 ACC@1: 77.68% ACC@5: 97.76%: 100%|| 40/40 [00:01<00:00, 21.13i
Test Epoch: [5/10] Loss: 0.6403 ACC@1: 81.09% ACC@5: 98.94%: 100%|| 79/79 [00:02<00:00, 32.45it
Train Epoch: [6/10] Loss: 0.7368 ACC@1: 77.44% ACC@5: 97.72%: 100%|| 40/40 [00:02<00:00, 19.02i
Test Epoch: [6/10] Loss: 0.6030 ACC@1: 81.75% ACC@5: 98.97%: 100%|| 79/79 [00:02<00:00, 29.37it
Train Epoch: [7/10] Loss: 0.6951 ACC@1: 78.12% ACC@5: 98.26%: 100%|| 40/40 [00:02<00:00, 19.25i
```

```
Test Epoch: [7/10] Loss: 0.5822 ACC@1: 81.78% ACC@5: 98.97%: 100%|| 79/79 [00:02<00:00, 31.88it
Train Epoch: [8/10] Loss: 0.6806 ACC@1: 78.10% ACC@5: 98.20%: 100%|| 40/40 [00:02<00:00, 18.38i
Test Epoch: [8/10] Loss: 0.5606 ACC@1: 82.40% ACC@5: 99.01%: 100%|| 79/79 [00:02<00:00, 28.84it
Train Epoch: [9/10] Loss: 0.6700 ACC@1: 78.38% ACC@5: 98.20%: 100%|| 40/40 [00:01<00:00, 21.43i
Test Epoch: [9/10] Loss: 0.5492 ACC@1: 82.41% ACC@5: 98.95%: 100%|| 79/79 [00:02<00:00, 30.98it
Train Epoch: [10/10] Loss: 0.6417 ACC@1: 79.42% ACC@5: 98.08%: 100%|| 40/40 [00:02<00:00, 19.04
Test Epoch: [10/10] Loss: 0.5354 ACC@1: 82.52% ACC@5: 99.01%: 100%|| 79/79 [00:02<00:00, 27.78i

Best top-1 accuracy with self-supervised learning:  82.52000000000001
```

6.2.1 Plot your Comparison

Plot the test accuracies between the baseline model (no pretraining) and same model pretrained with self-supervised learning.

```
In [33]: plt.plot(no_pretrain_results['test_acc@1'], label="Without Pretrain")
         plt.plot(pretrain_results['test_acc@1'], label="With Pretrain")
         plt.xlabel('Epochs')
         plt.ylabel('Accuracy')
         plt.title('Test Top-1 Accuracy')
         plt.legend()
         plt.show()
```



```
In [ ]:
```

DDPM

May 23, 2026

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

In [1]:
```
# Server setup cell.
import os
import sys

ASSIGNMENT_DIR = "/data/users/huanghao/a3/assignment3"
assert os.path.isdir(ASSIGNMENT_DIR), f"[!] Assignment directory not found: {ASSIGNMENT

if ASSIGNMENT_DIR not in sys.path:
    sys.path.append(ASSIGNMENT_DIR)

# Dataset download is optional here; uncomment if you need to fetch assignment dataset
# datasets_dir = os.path.join(ASSIGNMENT_DIR, "cs231n", "datasets")
# os.chdir(datasets_dir)
# !bash get_datasets.sh
# os.chdir(ASSIGNMENT_DIR)

print(f"Working directory: {os.getcwd()}")
```

```
Working directory: /data/users/huanghao/a3/assignment3
```

1 Denoising Diffusion Probabilistic Models

So far, we have explored discriminative models, which are trained to produce a labeled output. These range from straightforward image classification to sentence generation where the problem was still framed as classification, with labels in vocabulary space and a recurrence mechanism capturing multi-word labels. Now, we will expand our repertoire by building a generative model capable of creating novel images that resemble a given set of training images.

There are many types of generative models, including Generative Adversarial Networks (GANs), autoregressive models, normalizing flow models, and Variational Autoencoders (VAEs),

all of which can synthesize striking images. However, in 2020, Ho et al. introduced Denoising Diffusion Probabilistic Models (DDPMs) by combining diffusion probabilistic models with denoising score matching. This resulted in a generative model that is both simple to train and powerful enough to generate complex, high-quality images. The following provides a high-level overview of DDPMs. For more details, please refer to the course slides and the original DDPM paper [1].

2 Forward Process

Let $q(x_0)$ be the distribution of clean dataset images. We define a forward noising process as a Markov chain of small noising steps:

$$q(x_t|x_{t-1}) \sim N(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$$

where the stepwise variance $(\beta_1, ..., \beta_T)$ determines the noise schedule. Due to the properties of Gaussian distributions, we can express $q(x_t|x_0)$ in closed form as:

$$q(x_t|x_0) \sim N(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_t$. If the noise schedule $(\beta_1, ..., \beta_T)$ is set properly, the final distribution $q(x_T)$ becomes indistinguishable from pure Gaussian noise $N(0, I)$.

Recall that sampling from a Gaussian distribution $x \sim N(\mu, \sigma^2)$ is equivalent to computing $\sigma * \epsilon + \mu$ where $\epsilon \sim N(0, 1)$. Hence, sampling from $q(x_t|x_{t-1})$ or $q(x_t|x_0)$ is straightforward given x_{t-1} or x_0 respectively. Because of this, the forward process is simple and does not require learning.

3 Reverse Process

The reverse process reconstructs a clean image x_0 from pure noise x_T through multiple steps. Let $p(x_{t-1}|x_t)$ denote the reverse step of $q(x_t|x_{t-1})$. The first key insight is that learning to reverse each individual denoising step is easier than reversing the entire forward process in one go. In other words, learning $p(x_{t-1}|x_t)$ for each t is easier than directly learning $p(x_0|x_T)$.

However, learning $p(x_{t-1}|x_t)$ is still challenging. Although $q(x_t|x_{t-1})$ is Gaussian, $p(x_{t-1}|x_t)$ could take any complex form and is almost certainly not Gaussian. Modeling and sampling from an arbitrary distribution is significantly harder than working with a simple parametric distribution like a Gaussian.

The second key insight is that if the stepwise noise β_t in the forward process is small enough, then the reverse step $p(x_{t-1}|x_t)$ is also close to a Gaussian distribution. Thus, we only need to estimate its mean and variance. In practice, setting the variance of $p(x_{t-1}|x_t)$ to match β_t (the same as in the forward step) works well. Consequently, learning the reverse process reduces to learning the mean $\mu(x_t, t; \theta)$ where θ represents the parameters of a neural network.

4 Denoising Objective

Generative models are optimized by minimizing the expected negative log-likelihood $\mathbb{E}[-\log p_\theta(x_0)]$ of the dataset samples. The likelihood of each sample can be expressed as: $p_\theta(x_0) = p(x_T) \prod_{t=1}^{T} p(x_{t-1}|x_t)$. Since this objective is intractable in many cases, various classes of generative models optimize the variational lower bound on the negative log-likelihood.

Ho et al. demonstrated that this objective is equivalent to minimizing the following denoising loss

$$\mathbb{E}_{t,x_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)\|^2\right]$$

where t is uniform between 1 and T, x_0 is clean sample, ϵ is sampled from standard gaussian $N(0, I)$, and ϵ_θ is a neural network model trained to predict the noise ϵ from the input noisy sample $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$. In other words, ϵ_θ learns to denoise the input noisy image. Note that this is equivalent to predicting the clean sample, since the noise can be recovered from the noisy image and the clean sample following the equation $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$.

[1] Denoising Diffusion Probabilistic Models. Jonathan Ho, Ajay Jain, Pieter Abbeel. Link

5 In This Notebook...

We will implement and train a DDPM model to generate small 32 x 32 emoji images conditioned on text prompts. First, we will implement the forward noising process based on Eq. (4) of the paper [1]. Then we will build a UNet model that takes x_t and t as inputs (optionally with other conditioning like text-prompt) and outputs a tensor of the same shape as x_t. Finally, we will implement the denoising objective and train our DDPM model.

We use the text encoder from a pretrained CLIP[2] model to encode input text into a 512-dimensional vector. To speed up training, we've already pre-encoded the text data from the training set.

[2] Learning transferable visual models from natural language supervision. Radford et. al. Link

In [2]:
```
# CLIP is already installed in the server environment.
import clip
```

In [3]:
```
import sys
import types
import importlib

if "imp" not in sys.modules:
    imp = types.ModuleType("imp")
    imp.reload = importlib.reload
    sys.modules["imp"] = imp

%load_ext autoreload
%autoreload 2

import numpy as np
import torch
import random
import matplotlib.pyplot as plt
import torchvision.utils as tv_utils
from cs231n.emoji_dataset import EmojiDataset
from cs231n.gaussian_diffusion import GaussianDiffusion
```

```
        def rel_error(x, y):
            """Returns relative error."""
            return np.max(np.abs(x - y) / (np.maximum(1e-10, np.abs(x) + np.abs(y))))
```

```
In [4]: # First, let's load and visualize the dataset.
        # Each sample of the dataset is a tuple: (image, {"text_emb": <tensor>, "text": <string
        # We will use a pretrained text-encoder to encode text into an embedding vector.
        # We have pre-encoded the dataset texts into embeddings for faster training.
        image_size = 32
        dataset = EmojiDataset(image_size)
```

```
emoji_data.npz already downloaded.
text_embeddings.pt already downloaded.
```

```
In [5]: def visualize_samples(dataset, num_samples=25, grid_size=(5, 5)):
            # Randomly sample indices
            indices = random.sample(range(len(dataset)), num_samples)
            samples = [dataset[i] for i in indices]

            # Inspect one sample
            img_shape = list(samples[0][0].shape)
            emb_shape = list(samples[0][1]["text_emb"].shape)
            print(f"One sample: (image: {img_shape}, {{ \"text_emb\": {emb_shape}, \"text\": st

            # Extract images and texts
            images = torch.stack([sample[0] for sample in samples])  # Stack images
            texts = [sample[1]["text"] for sample in samples]  # Extract text descriptions

            # Create a grid of images
            grid_img = tv_utils.make_grid(images, nrow=grid_size[1], padding=2)

            # Convert to numpy for plotting
            grid_img = grid_img.permute(1, 2, 0).numpy()

            # Plot the images
            fig, ax = plt.subplots(figsize=(10, 10))
            ax.imshow(grid_img)
            ax.axis("off")

            # Add text annotations
            grid_w, grid_h = grid_size
            img_w, img_h = grid_img.shape[1] // grid_w, grid_img.shape[0] // grid_h

            for i, text in enumerate(texts):
                row, col = divmod(i, grid_w)
                x, y = col * img_w, row * img_h
                ax.text(x+5, y+5, text[:30], fontsize=8, color='white', bbox=dict(facecolor='bl
```

```
            plt.show()

        visualize_samples(dataset)
```

```
One sample: (image: [3, 32, 32], { "text_emb": [512], "text": string })
```

5.1 q_sample

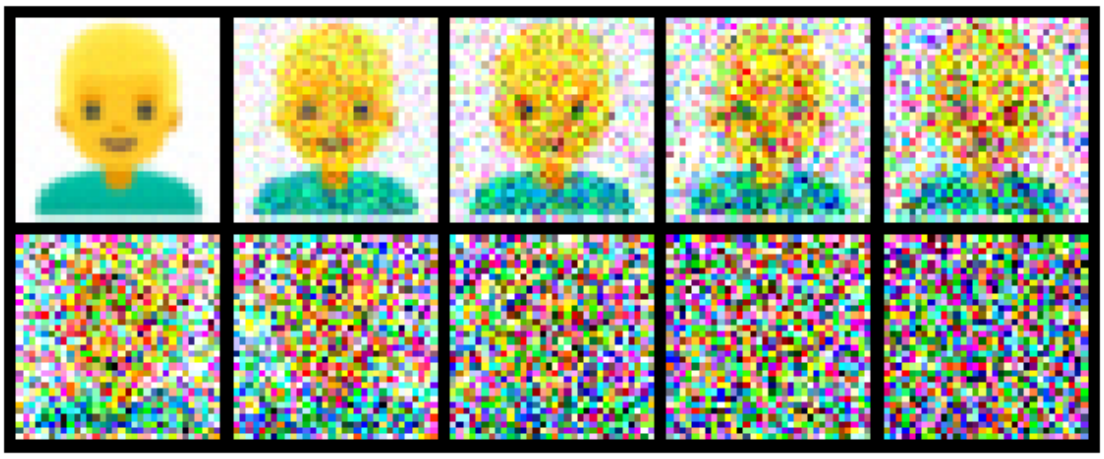
Now we will define the forward noising process. Read through the GaussianDiffusion class in `cs231n/gaussian_diffusion.py`. Consult the original DDPM paper[1] for the equations. Implement `q_sample` method and test it below. You should see zero relative error.

In [6]:
```
# Test GaussianDiffusion.q_sample method
sz = 2
b = 3

diffusion = GaussianDiffusion(
      model=None,
      image_size=sz,
      timesteps=1000,
      beta_schedule="sigmoid",
)

t = torch.tensor([0, 300, 999]).long()
x_start = torch.linspace(-0.9, 0.6, b*3*sz*sz).view(b, 3, sz, sz)
noise = torch.linspace(-0.7, 0.8, b*3*sz*sz).view(b, 3, sz, sz)
x_t = diffusion.q_sample(x_start, t, noise)

expected_x_t = np.array([
    [
        [[-0.9119949, -0.86840147], [-0.8248081, -0.7812148]],
        [[-0.7376214, -0.694028], [-0.65043473, -0.6068413]],
        [[-0.563248, -0.51965463], [-0.47606122, -0.43246788]],
    ],
    [
        [[-0.42800453, -0.37039882], [-0.31279305, -0.2551873]],
        [[-0.19758154, -0.1399758], [-0.08237009, -0.024764337]],
        [[0.032841414, 0.090447165], [0.14805292, 0.20565866]],
    ],
    [
        [[0.32864183, 0.37152246], [0.41440308, 0.45728368]],
        [[0.50016433, 0.5430449], [0.5859255, 0.6288062]],
        [[0.67168677, 0.7145674], [0.757448, 0.8003287]],
    ],
]).astype(np.float32)

# Should see zero relative error
print("x_t error: ", rel_error(x_t.numpy(), expected_x_t))
```

```
x_t error:  0.0
```

In [7]:
```
# Let's visualize the noisy images at various timesteps.
diffusion = GaussianDiffusion(
      model=None,
      image_size=image_size,
      timesteps=1000,
)

B = 10
```

```
img = dataset[770][0]  # 3 x H x W
x_start = img[None].repeat(B, 1, 1, 1)  # B x 3 x H x W
noise = torch.randn_like(x_start)  # B x 3 x H x W
t = torch.linspace(0, 1000-1, B).long()

x_start = diffusion.normalize(x_start)
x_t = diffusion.q_sample(x_start, t, noise)
x_t = diffusion.unnormalize(x_t).clamp(0, 1)
grid_img = tv_utils.make_grid(x_t, nrow=5, padding=2)
grid_img = grid_img.permute(1, 2, 0).cpu().numpy()
fig, ax = plt.subplots(figsize=(10, 10))
ax.imshow(grid_img)
ax.axis("off")
plt.show()
```



A diffusion model can be trained to predict either the clean image or the noise, as one can be derived from the other (explained in 'Denoising Objective' section above). Implement `predict_start_from_noise` and `predict_noise_from_start` methods and test them below. You should see relative error less than 1e-5.

In [8]:
```
# Test `predict_noise_from_start` and `predict_start_from_noise`
sz = 2
b = 3

diffusion = GaussianDiffusion(
      model=None,
      image_size=sz,
      timesteps=1000,
      beta_schedule="sigmoid",
)

t = torch.tensor([1, 300, 998]).long()
```

```
        x_start = torch.linspace(-0.91, 0.67, b*3*sz*sz).view(b, 3, sz, sz)
        noise = torch.linspace(-0.73, 0.81, b*3*sz*sz).view(b, 3, sz, sz)
        x_t = diffusion.q_sample(x_start, t, noise)

        pred_noise = diffusion.predict_noise_from_start(x_t, t, x_start)
        pred_x_start = diffusion.predict_start_from_noise(x_t, t, noise)

        # Should relative errors around 1e-5 or less
        print("noise error: ", rel_error(pred_noise.numpy(), noise.numpy()))
        print("x_start error: ", rel_error(pred_x_start.numpy(), x_start.numpy()))
```

```
noise error:  1.0600407e-06
x_start error:  1.8902663e-06
```

5.2 UNet Model

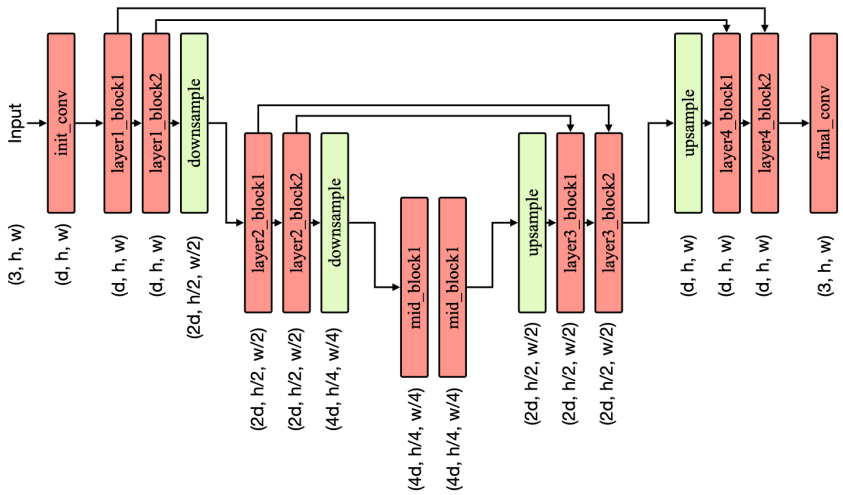
Now that we have defined the forward process, let's define the UNet model for denoising the input image. UNet is a neural network architecture designed for image-to-image tasks like segmentation, style transfer, etc. It consists of an encoder (or downsampling module) that transforms the input image into hierarchical features with decreasing spatial resolution and increasing feature dimensions. The decoder (or upsampling module) then upsamples the features by progressively restoring the spatial resolution, mirroring the encoder's structure. At each decoder layer, features from the corresponding encoder layer are concatenated, providing a direct pathway for finer details. This approach reduces the burden on the bottleneck layers, allowing them to focus on capturing high-level representations rather than memorizing fine details.

We will use UNet in this case because both our input and output are aligned images with same dimensions: C x H x W. Each ResNet block in the UNet will also take an additional input vector called context for conditioning. We will generate the context vector by encoding the diffusion timestep and text-prompt.

Run the cell below to get a rough outline of the UNet architecture. Each red box represents a ResNet block containing 2 or 3 convolutional layers that maintain the spatial resolution of the feature maps. The context vector input to every ResNet block is ommitted for clarity. The shape written below each box indicates the output tensor shape after that block. Additional arrows illustrate the skip connections, which enable the U-Net to preserve fine-grained details in the output. For example, the output of `layer1_block1` with shape (d, h, w) will be concatenated with the output of `layer4_block1`, also with shape (d, h, w), before being passed to `layer4_block2`. Therefore, `layer4_block2` will receive an input of shape (2*d, h, w).

In [9]:
```
from IPython.display import Image
Image(os.path.join(ASSIGNMENT_DIR, 'unet.png'))
```

Out[9]:



Implement the `Unet.__init__` method in `cs231n/unet.py` to define the upsampling and downsampling blocks of the UNet model, and then test it below. If your implementation is correct, you should not see any error. Calling `Unet(dim=d, condition_dim=condition_dim, dim_mults=(2,4))` should successfully create a UNet model corresponding to the architecture shown in the figure above.

```
In [10]: from cs231n.unet import Unet, ResnetBlock, Downsample, Upsample

         dim = 2
         condition_dim = 4
         dim_mults = (1, 2, 4)
         unet = Unet(dim=dim, condition_dim=condition_dim, dim_mults=dim_mults)


         # Check number of layers
         assert len(unet.downs) == len(dim_mults), "Number of Unet downsampling blocks is wrong
         assert len(unet.ups) == len(dim_mults), "Number of Unet upsampling blocks is wrong."


         # Check layers
         try:
           expected_downs_dims = [2, 2, 8, 2, 2, 8, 2, 2, 8, 2, 2, 8, 4, 4, 8, 4, 4, 8]
           downs_dims = [
               d for m in unet.downs for d in [
                   m[0].dim, m[0].dim_out, m[0].context_dim, m[1].dim, m[1].dim_out, m[1].conte
               ]
           ]
           assert downs_dims == expected_downs_dims, "Dimensions don't match"
```

```
except Exception as e:
  raise RuntimeError("Downsampling blocks wrongly configured") from e


try:
  expected_ups_dims = [8, 4, 8, 8, 4, 8, 4, 2, 8, 4, 2, 8, 4, 2, 8, 4, 2, 8]
  ups_dims = [
      d for m in unet.ups for d in [
          m[1].dim, m[1].dim_out, m[1].context_dim, m[2].dim, m[2].dim_out, m[2].conte
      ]
  ]
  assert ups_dims == expected_ups_dims, "Dimensions don't match"
except Exception as e:
  raise RuntimeError("Upsampling blocks wrongly configured") from e

# Check number of parameters
num_params = sum(p.numel() for p in unet.parameters())
expected_num_params = 6499
assert num_params == expected_num_params, "Unet model creation is wrong!"
```

Fill in `Unet.forward` method in `cs231n/unet.py` and test it below. For now, don't worry about `Unet.cfg_forward` method. You should see relative error less than 1e-6.

In [11]:
```
np.random.seed(231)
torch.manual_seed(231)

dim = 4
condition_dim = 4
dim_mults = (2, 4)
unet = Unet(dim=dim, condition_dim=condition_dim, dim_mults=dim_mults)
unet.eval()

b = 2
h = w = 4
inp_x = torch.randn(b, 3, h, w)
inp_text_emb = torch.randn(b, condition_dim)
inp_t = torch.tensor([8, 25]).long()
out = unet.forward(x=inp_x, time=inp_t,
                   model_kwargs={"text_emb": inp_text_emb}).detach().numpy()

expected_out = np.array(
      [[[[ 0.3530089,   0.49302575,  0.55152094,  0.20567769],
         [ 1.1295223,   0.05811183,  0.2143015,   0.23898259],
         [ 0.44839963,  1.0291728,   0.18519637,  0.21956968],
         [ 0.7035912,   0.716347,    0.9166326,  -0.11552593]],
        [[ 0.04585427, -0.26847702,  0.08526254,  0.2396591 ],
         [ 0.2858743,  -0.29726404,  0.02462834,  0.2350314 ],
         [ 0.3121434,   0.3962962,   0.65541625, -0.300183  ],
```

```
                [ 0.3409412,   0.35369393,  0.37445623,  0.10297439]],
               [[ 0.49440542,  0.72878885,  0.6669705,   0.56822944],
                [ 1.0084232,   0.19204213,  0.61240125,  0.575651  ],
                [ 0.6452671,   0.91168964,  0.6458106,   0.25952405],
                [ 0.7147298,   0.7707498,   0.88941365,  0.35422024]]],
              [[[ 0.3186316,  -0.17632714,  0.440247,    0.05124736],
                [ 0.6866561,   0.3019496,   0.18366373,  0.42941707],
                [-0.09776017,  0.55849403,  0.30689716,  0.12954405],
                [ 0.3667726,   0.740231,    0.43682802,  0.08275545]],
               [[-0.29714173, -0.28515863, -0.02289355,  0.05637485],
                [-0.10230345,  0.182563,    0.8107456,   0.18209176],
                [ 0.57062995,  0.07061654,  0.2864037,   0.17969263],
                [ 0.31491554,  0.5652035,   0.3810383,   0.05889529]],
               [[ 0.14559639,  0.36408228,  0.5657894,   0.24066216],
                [ 0.4431491,   0.38140664,  1.1457629,   0.40314198],
                [ 0.6673274,   0.5922724,   0.44549578,  0.5544277 ],
                [ 0.5405652,   0.88729244,  0.66340685,  0.23414826]]]])
```

```
print("forward error: ", rel_error(out, expected_out))
```

```
forward error:  6.522376763496169e-06
```

6 p_losses

Now that we have model implementation done, let's write the DDPM's denoising training step. As mentioned before, optimizing the denoising loss is equivalent to minimizing the expected negative log likelihood of the dataset. Complete the `GaussianDiffusion.p_losses` method in `cs231n/gaussian_diffusion.py` and test it below. You should see relative error less than 1e-6 .

In [12]:
```
np.random.seed(231)
torch.manual_seed(231)

dim = 4
condition_dim = 4
dim_mults = (2, 4)
unet = Unet(dim=dim, condition_dim=condition_dim, dim_mults=dim_mults)
unet.eval()

h = w = 4
b = 3
diffusion = GaussianDiffusion(
      model=unet,
      image_size=h,
      timesteps=1000,
      beta_schedule="sigmoid",
      objective="pred_x_start",
)
```

```
        inp_x = torch.randn(b, 3, h, w)
        inp_model_kwargs = {"text_emb": torch.randn(b, condition_dim)}
        out = diffusion.p_losses(inp_x, inp_model_kwargs)
        expected_out = 30.0160

        print("forward error: ", rel_error(out.item(), expected_out))
```

```
forward error:  7.432147198281673e-07
```

6.1 p_sample

There is one final ingredient remaining now. DDPM generates samples by iteratively performing the reverse process. Each iteration of this reverse process involves sampling from $p(x_{t-1}|x_t)$. Open `cs231n/gaussian_diffusion.py` and implement `p_sample` method by following Equation (6) from the paper. This equation describes sampling from the posterior of the forward process, conditioned on x_t and x_0, where x_0 can be derived from the denoising model's output. We have already implemented `sample` method that iteratively calls `p_sample` to generate images from input texts.

Test your implementation of `p_sample` below, you should see relative errors less than 1e-6.

In [13]:
```
np.random.seed(231)
torch.manual_seed(231)

dim = 4
condition_dim = 4
dim_mults = (2,)
unet = Unet(dim=dim, condition_dim=condition_dim, dim_mults=dim_mults)
unet.eval()

h = w = 4
b = 1
inp_x_t = torch.randn(b, 3, h, w)
inp_model_kwargs = {"text_emb": torch.randn(b, condition_dim)}
t = 231

# test 1
diffusion = GaussianDiffusion(
      model=unet,
      image_size=h,
      timesteps=1000,
      beta_schedule="sigmoid",
      objective="pred_x_start",
)
out = diffusion.p_sample(inp_x_t, t, inp_model_kwargs).detach().numpy()

expected_out = np.array(
```

```
        [[[[ 1.1249855,   0.098786,   -0.674154,    1.3266845 ],
             [-0.21832949,  0.4874724,   1.0712924,  -0.7355726 ],
             [-0.19198017,  1.0347139,   0.03363481,  0.47500697],
             [-1.0178545,  -0.52259684, -0.11972852, -0.02105119]],
            [[ 0.06438226, -1.5796007,   0.43629852, -0.67392266],
             [-1.1564192,  -1.7009112,   0.878653,   -1.2563533 ],
             [-0.45194352,  1.0274173,   0.62859684, -0.49643248],
             [-0.18070872,  0.51325583, -0.74465716, -0.5262963 ]],
            [[ 0.76330817,  1.5878401,   1.9983996,   0.6618041 ],
             [ 0.16782297,  0.5638586,  -0.8150751,   1.9507835 ],
             [ 0.7401889,   1.2696184,   1.8468435,  -1.4796975 ],
             [ 0.6636643,  -0.83933365,  0.78857577,  0.13521622]]]])
    print("forward error: ", rel_error(out, expected_out))

    # test 2
    diffusion = GaussianDiffusion(
          model=unet,
          image_size=h,
          timesteps=1000,
          beta_schedule="cosine",
          objective="pred_noise",
    )
    out = diffusion.p_sample(inp_x_t, t, inp_model_kwargs).detach().numpy()

    expected_out = np.array(
        [[[[ 1.1139543,   0.11015256, -0.7337392,   1.3292073 ],
             [-0.286727,    0.48420003,  1.0737748,  -0.764173  ],
             [-0.22848499,  1.0443672,   0.07089197,  0.50183666],
             [-1.0453997,  -0.5474187,  -0.06630269, -0.11730157]],
            [[ 0.10919069, -1.5161378,   0.38542387, -0.6693473 ],
             [-1.0730051,  -1.6837747,   0.87086374, -1.2265388 ],
             [-0.48162907,  1.0328836,   0.6336425,  -0.47346604],
             [-0.11769794,  0.5335539,  -0.71324545, -0.47889465]],
            [[ 0.8311781,   1.6457081,   1.9775101,   0.563749  ],
             [ 0.25127694,  0.57862943, -0.8148284,   1.8744429 ],
             [ 0.75546896,  1.1215659,   1.9582558,  -1.535511  ],
             [ 0.62724066, -0.8814953,   0.77542645,  0.12667023]]]])
    print("forward error: ", rel_error(out, expected_out))
```

```
forward error:  6.96344541990084e-08
forward error:  3.3032469825584936e-08
```

6.2 Training

We have all ingredients needed for DDPM training and we can train the model on our Emoji dataset. You don't have to code anything here but we encourage you to look at the training code at `cs231n/ddpm_trainer.py`.

For the rest of the notebook, we will use a pretrained model from `cs231n/exp/pretrained`, which is already trained for many iterations on this dataset. You can also train your own model on the server GPU if available by changing the `results_folder`. Note that training long enough to get good generations may still take many hours.

In [14]:
```
from cs231n.ddpm_trainer import Trainer

dim = 48
image_size = 32
results_folder = os.path.join(ASSIGNMENT_DIR, "cs231n", "exp", "pretrained")
condition_dim = 512

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

model = Unet(
    dim=dim,
    dim_mults=(1, 2, 4, 8),
    condition_dim=condition_dim,
)
print("Number of parameters:", sum(p.numel() for p in model.parameters()))

diffusion = GaussianDiffusion(
    model,
    image_size=image_size,
    timesteps=100,  # number of diffusion steps
    objective="pred_noise",  # "pred_x_start" or "pred_noise"
)

dataset = EmojiDataset(image_size)

trainer = Trainer(
    diffusion,
    dataset,
    device,
    train_batch_size=256,
    weight_decay=0.0,
    train_lr=1e-3,
    train_num_steps=50000,
    results_folder=results_folder,
)

# You are not required to train it yourself.
# trainer.train()
```

```
Number of parameters: 12444963
emoji_data.npz already downloaded.
text_embeddings.pt already downloaded.
```

```
In [15]: # Instead, we will load a pretrained model.
         trainer.download_pretrained()
         trainer.load(70000)
```

```
Downloading.../data/users/huanghao/a3/assignment3/cs231n/exp/pretrained/model-70000.pt
Download complete.
loading model from /data/users/huanghao/a3/assignment3/cs231n/exp/pretrained/model-70000.pt.


/data/users/huanghao/miniconda3/envs/cs224n-a3/lib/python3.8/site-packages/torch/_utils.py:776
  return self.fget.__get__(instance, owner)()
/data/users/huanghao/miniconda3/envs/cs224n-a3/lib/python3.8/site-packages/torch/cuda/__init__
  warnings.warn("Can't initialize NVML")
```

```
In [16]: # Helper function to get CLIP text embedding during inference time.
         from cs231n.emoji_dataset import ClipEmbed
         clip_embedder = ClipEmbed(device)
         def get_text_emb(text):
             return trainer.ds.embed_new_text(text, clip_embedder)

         # Helper function to visualize generations.
         def show_images(img):
             # img: B x T x 3 x H x W
             plt.figure(figsize=(10, 10))
             img2 = img.clamp(0, 1).permute(0, 3, 1, 4, 2).flatten(0, 1).flatten(1, 2).cpu().nu
             plt.imshow(img2)
             plt.axis('off')

             plt.show()
```

```
100%|| 338M/338M [00:05<00:00, 60.1MiB/s]
```

6.3 Sampling

Run the cell below to visualize emoji generations conditioned on a text prompt. Feel free to modify the prompt to explore different generations. Since our emoji dataset is quite small, it is insufficient to train a fully generalizable text-to-image model. Because of that, generations for unseen prompts may be poor or may not be faithful to the input text (this may also happen for seen examples but less common).

For faster sampling, use a GPU on the server if one is available. If you switch Python environments or kernels, rerun the notebook setup cells first.

```
In [17]: # text = "crying face"  # seen example, good generations
         text = "face with cowboy hat"  # seen example, good generations
         # text = "crying face with cowboy hat"  # unseen example, bad generations
         text_emb = get_text_emb(text)
         text_emb = text_emb[None].expand(5, -1).to(device)
```

```
        img = trainer.diffusion_model.sample(
            batch_size=5,
            model_kwargs={"text_emb": text_emb},
            return_all_timesteps=True
        )
        show_images(img[:, ::20])
```

```
sampling loop time step:   0%|          | 0/100 [00:00<?, ?it/s]
```

6.4 Classifier Free Guidance

Generative models are typically evaluated on fidelity (i.e., the quality or realism of the generated samples) and diversity (the variability or coverage of the sample space). For conditional generative models, fidelity additionally refers to how faithfully the generated samples adhere to the input conditions. These two metrics are often in tension with each other, leading to a trade-off

between them. Ho et al. introduced a simple technique called classifier-free guidance [3], which allows explicit control over this trade-off.

In classifier-free guidance, during the training of a conditional diffusion model $\epsilon_\theta(x_t, t, c)$, the condition c is randomly dropped (i.e. replaced with $c = \phi$) with some probability (typically 0.1 to 0.2). During each denoising step of sampling, the prediction is updated as:

$$\epsilon_\theta(x_t, t, c) \leftarrow (w+1)\epsilon_\theta(x_t, t, c) - w\epsilon_\theta(x_t, t, \phi)$$

where w is a positive scalar (the guidance scale). In other words, we perform two predictions during each denoising step, one conditional and one unconditional, and combine them linearly to favor the conditional generation. w is a hyperparameter that is tuned to optimize a model-specific evaluation metric. A higher w makes the generations more faithful to the condition but tends to reduce their diversity.

[3] Classifier-Free Diffusion Guidance. Jonathan Ho, Tim Salimans. Link

Implement the classifier-free guidance in `Unet.cfg_forward` method in `cs231n/unet.py` and test it below. You should see relative error less than 1e-6.

In [18]:
```
np.random.seed(231)
torch.manual_seed(231)

dim = 4
condition_dim = 4
dim_mults = (2, 4)
unet = Unet(dim=dim, condition_dim=condition_dim, dim_mults=dim_mults)
unet.eval()

b = 2
h = w = 4
inp_x = torch.randn(b, 3, h, w)
inp_text_emb = torch.randn(b, condition_dim)
inp_t = torch.tensor([8, 25]).long()
out = unet.forward(x=inp_x, time=inp_t,
                   model_kwargs={"text_emb": inp_text_emb, "cfg_scale": 2.31}
                   ).detach().numpy()
expected_out = np.array(
      [[[[ 0.3063889,   0.49175858,  0.53406465,  0.17793494],
         [ 1.0605793,   0.07291106,  0.26333052,  0.2807781 ],
         [ 0.47007525,  1.0033083,   0.15596265,  0.21308547],
         [ 0.6635294,   0.69716704,  0.7332697,  -0.14285703]],
        [[-0.14788617, -0.17531544,  0.06930217,  0.24703366],
         [ 0.2998472,  -0.25145096,  0.06968095,  0.28924388],
         [ 0.311122,    0.35599577,  0.6346719,  -0.30859607],
         [ 0.3706389,   0.33177835,  0.29829532,  0.11159261]],
        [[ 0.34566414,  0.79019654,  0.6489645,   0.5683719 ],
         [ 0.98117805,  0.176887,    0.6220584,   0.6494926 ],
         [ 0.66813314,  0.90341353,  0.65361,     0.21999073],
         [ 0.73401093,  0.75344837,  0.71971107,  0.32417077]]],
       [[[ 0.46322435, -1.3915588,   1.0951779,   0.37392217],
         [ 0.6877824,   0.14802843, -0.7159017,   0.22476494],
```

```
                [-0.28032526,  0.50201464,  0.0972808,   0.23250426],
                [ 0.38375473,  0.86657,     0.4424622,   0.2161349 ]],
               [[-0.34920466, -0.48507357,  0.06529156,  0.29454708],
                [-0.62153226,  0.0083245,   1.7356712,  -0.0690068 ],
                [ 0.7745638,  -0.11548355,  0.237822,    0.22680345],
                [ 0.06337905,  0.7001358,   0.36441565,  0.17163567]],
               [[ 0.16040441, -0.2551211,   0.8887521,   0.11380547],
                [-0.12803352,  0.2783994,   1.4620805,   0.19044566],
                [ 0.9112923,   0.75666,     0.3765695,   0.66629255],
                [ 0.3461262,   1.0881201,   0.6392708,   0.31749305]]]])
```

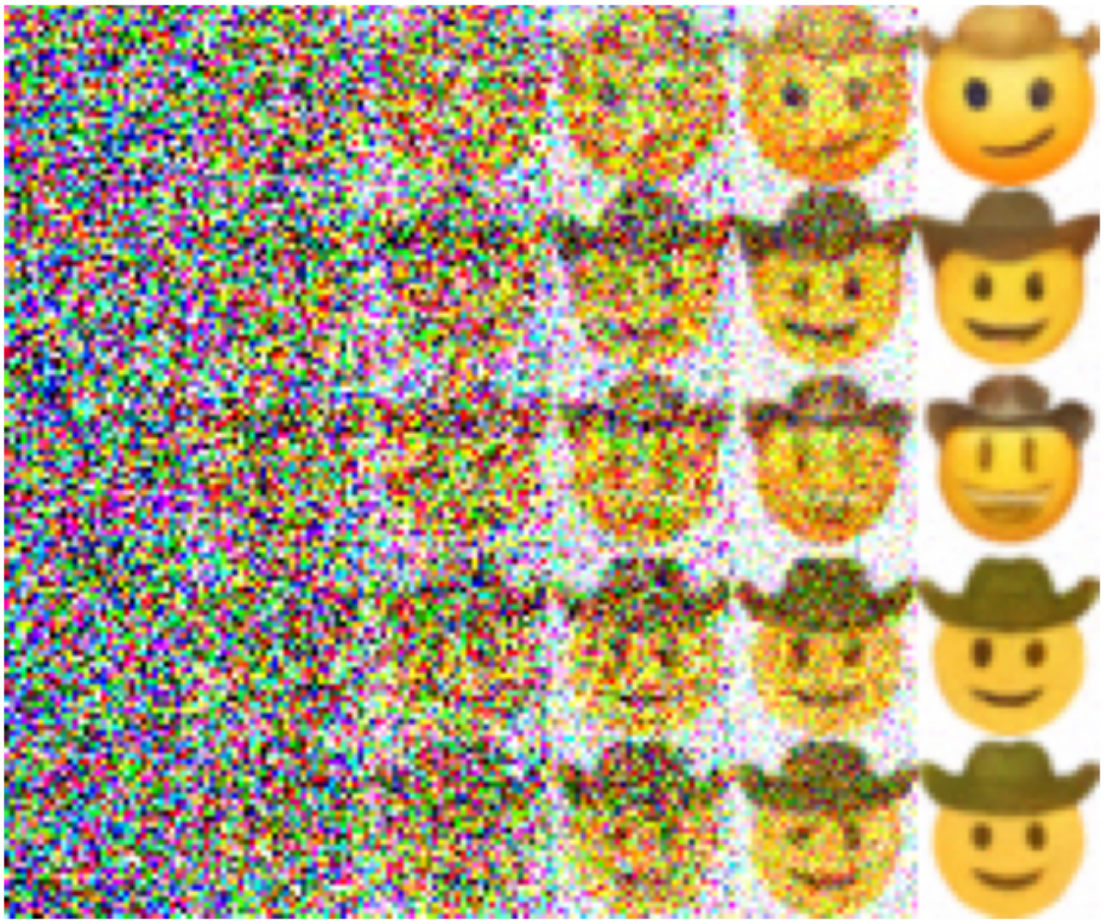
```
In [ ]: print("forward error: ", rel_error(out, expected_out))
```

```
forward error:  5.346979688215397e-05
```

Run the cell below to visualize emoji generations using classifier-free guidance. Feel free to modify the "cfg_scale" parameter value as well. As mentioned earlier, since our model does not generalize well, you may or may not observe faithful generations even with a high guidance scale.

```
In [19]: # text = "crying face"  # seen example, good generations
         text = "face with cowboy hat"  # seen example, good generations
         # text = "crying face with cowboy hat"  # unseen example, bad generations
         text_emb = get_text_emb(text)
         text_emb = text_emb[None].expand(5, -1).to(device)


         img = trainer.diffusion_model.sample(
             batch_size=5,
             model_kwargs={"text_emb": text_emb, "cfg_scale": 1},
             return_all_timesteps=True
         )
         show_images(img[:, ::20])
```

```
sampling loop time step:   0%|          | 0/100 [00:00<?, ?it/s]
```



In [ ]:

CLIP_DINO

May 23, 2026

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

In [17]:
```
# Local server setup
import os
import sys

ASSIGNMENT_DIR = "/data/users/huanghao/a3/assignment3"
assert os.path.isdir(ASSIGNMENT_DIR), f"[!] Assignment directory not found: {ASSIGNME

os.chdir(ASSIGNMENT_DIR)
if ASSIGNMENT_DIR not in sys.path:
    sys.path.append(ASSIGNMENT_DIR)

print(f"Working directory: {os.getcwd()}")
```

```
Working directory: /data/users/huanghao/a3/assignment3
```

In [18]:
```
# Download the COCO dataset locally if it doesn't already exist.
# You should already have this dataset from the previous notebook.
# Uncomment the following lines only if needed.
# %cd /data/users/huanghao/a3/assignment3/cs231n/datasets
# !bash get_coco_dataset.sh
# %cd /data/users/huanghao/a3/assignment3
```

In [19]:
```
# Optional: install extra packages in the active notebook kernel if needed.
# %pip install ftfy regex tqdm decord

# CLIP should already be available in the configured assignment environment.
# If it is missing, uncomment the line below and rerun this cell.
# %pip install git+https://github.com/openai/CLIP.git
```

1 State-of-the-Art Pretrained Image Models

In the previous exercise, you learned about SimCLR and how contrastive self-supervised learning can be used to learn meaningful image representations. In this notebook, we will explore two more recent models that also aim to learn high-quality visual representations and have demonstrated strong and robust performance on a variety of downstream tasks.

First, we will examine the CLIP model. Like SimCLR, CLIP uses a contrastive learning objective, but instead of contrasting two augmented views of the same image, it contrasts two different modalities: text and image. To train CLIP, OpenAI collected a large dataset of ~400M image-text pairs from the internet, including sources like Wikipedia and image alt text. The resulting model learns rich, high-level image features and has achieved impressive zero-shot performance on many vision benchmarks.

Next, we will explore DINO, a self-supervised learning method for vision tasks that applies contrastive learning in a self-distillation framework with multi-crop augmentation strategy. The authors showed that the features learned by DINO ViTs are fine-grained and semantically rich with explicit information about the semantic segmentation of the image.

2 CLIP

As explained above, CLIP's training objective incorporates both text and images, building upon the principles of contrastive learning. Consider this quote from the SimCLR notebook: >The goal of the contrastive loss is to maximize agreement between the final vectors $z_i = g(h_i)$ and $z_j = g(h_j)$.

Similarly, CLIP is trained to maximize agreement between two vectors. However, because these vectors come from different modalities, CLIP uses two separate encoders: a transformer-based Text Encoder and a Vision Transformer (ViT)-based Image Encoder. Note that some smaller, more efficient versions of CLIP use a ResNet as the Image Encoder instead of a ViT.

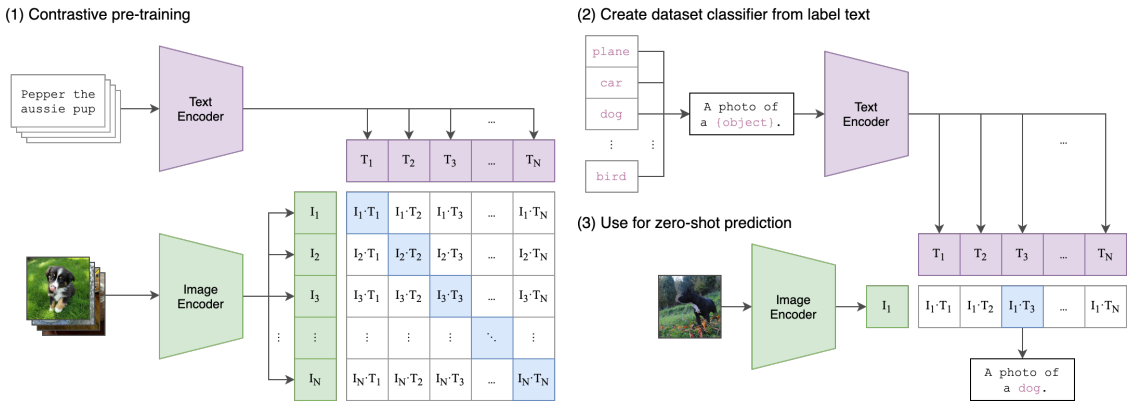
Run the cell below to visualize the training and inference pipeline of CLIP.

During the pretraining phase, each batch consists of multiple images along with their corresponding captions. Each image is independently processed by an Image Encoder—typically a visual model like a Vision Transformer (ViT) or a Convolutional Neural Network (ConvNet)—which produces an image embedding I_n. Likewise, each caption is independently processed by a Text Encoder to generate a corresponding text embedding T_n. Next, we compute the pairwise similarities between all image-text combinations, meaning each image is compared with every caption, and vice versa. The training objective is to maximize the similarity scores along the diagonal of the resulting similarity matrix – that is, the scores for the matching image-caption pairs (I_n, T_n). Through backpropagation, the model learns to assign higher similarity scores to true matches than to mismatched pairs.

Through this setup, CLIP effectively learns to represent images and texts in a shared latent space. In this space, semantic concepts are encoded in a modality-independent way, enabling meaningful cross-modal comparisons between visual and textual inputs.

```
In [20]: from IPython.display import Image as DisplayImage
         DisplayImage(os.path.join(ASSIGNMENT_DIR, 'CLIP.png'))
```

```
Out[20]:
```



Inline Question 1 -

Why does CLIP's learning depend on the batch size? If the batch size is fixed, what strategy can we use to learn rich image features?

Your Answer : CLIP uses a contrastive loss where, in each batch, the matching image-text pair is treated as the positive pair and all other image-text combinations in the batch are treated as negatives. Therefore, a larger batch provides more negative examples, making the contrastive task harder and giving the model a richer learning signal.

If the batch size is fixed, we can improve feature learning by using stronger and more diverse data augmentation, training for more epochs, or using pretrained encoders and fine-tuning.

3 Loading COCO dataset

We'll use the same captioning dataset you used to train your RNN captioning model, but instead of generating the captions lets see if we can match each image to the correct caption.

```
In [21]: import sys
         import types
         import importlib

         if "imp" not in sys.modules:
             imp = types.ModuleType("imp")
             imp.reload = importlib.reload
             sys.modules["imp"] = imp

         %load_ext autoreload
         %autoreload 2

         import time, os, json
         import numpy as np
         import matplotlib.pyplot as plt
         import torch
         import clip
         from tqdm import tqdm
```

```
from PIL import Image
from cs231n.clip_dino import *

def rel_error(x, y):
    """Returns relative error."""
    return np.max(np.abs(x - y) / (np.maximum(1e-10, np.abs(x) + np.abs(y))))
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```

In [22]:
```
from cs231n.coco_utils import load_coco_data, sample_coco_minibatch, decode_captions
from cs231n.image_utils import image_from_url
```

In [23]:
```
# Load COCO data from disk into a dictionary.
# this is the same dataset you used for the RNN captioning notebook :)
data = load_coco_data(pca_features=True)

# Print out all the keys and values from the data dictionary.
for k, v in data.items():
    if type(v) == np.ndarray:
        print(k, type(v), v.shape, v.dtype)
    else:
        print(k, type(v), len(v))
```

```
base dir  /data/users/huanghao/a3/assignment3/cs231n/datasets/coco_captioning
train_captions <class 'numpy.ndarray'> (400135, 17) int32
train_image_idxs <class 'numpy.ndarray'> (400135,) int32
val_captions <class 'numpy.ndarray'> (195954, 17) int32
val_image_idxs <class 'numpy.ndarray'> (195954,) int32
train_features <class 'numpy.ndarray'> (82783, 512) float32
val_features <class 'numpy.ndarray'> (40504, 512) float32
idx_to_word <class 'list'> 1004
word_to_idx <class 'dict'> 1004
train_urls <class 'numpy.ndarray'> (82783,) <U63
val_urls <class 'numpy.ndarray'> (40504,) <U63
```

In [24]:
```
# we're just using the loaded captions from COCO, so we need to decode them and get r
decoded_captions= []
for caption in data['val_captions']:
  caption = decode_captions(caption, data['idx_to_word'])\
    .replace('<START>', '')\
    .replace('<END>', '')\
    .replace('<UNK>', '')\
    .strip()
  decoded_captions.append(caption)
```

In [25]:
```
def to_uint8_rgb(img):
    img = np.asarray(img)
    if img.ndim == 2:
        img = np.stack([img] * 3, axis=-1)
    elif img.ndim == 3 and img.shape[2] == 4:
        img = img[:, :, :3]

    if img.dtype != np.uint8:
        if np.issubdtype(img.dtype, np.floating) and img.max() <= 1.0:
            img = img * 255.0
        img = np.clip(img, 0, 255).astype(np.uint8)

    return img

# lets get 10 examples
mask = np.array([135428, 122586, 122814, 133173, 176639, 163828,  98169,   6931,
        19488, 175760])
first_captions = [decoded_captions[elem] for elem in mask]

img_idxs = data['val_image_idxs'][mask]       # the images the captions refer to
first_images = [to_uint8_rgb(image_from_url(data['val_urls'][j])) for j in img_idxs]
print([img.dtype for img in first_images])
```

```
[dtype('uint8'), dtype('uint8'), dtype('uint8'), dtype('uint8'), dtype('uint8'), dtype('uint8'
```

In [26]:
```
for i, (caption, image) in enumerate(zip(first_captions, first_images)):
    plt.imshow(to_uint8_rgb(image))
    plt.axis('off')
    caption_str = caption
    plt.title(caption_str)
    plt.show()
```

a man holding a tennis racquet on a tennis court

two zebras walking away under the shade of a tree

a dog is standing in a kitchen sink

a hot dog with onions sitting on a

a young woman holding a baby with a teddy bear on her lap

buildings with a clock and along a street

a skateboarder is flying down a of stairs

a military jet flying overhead in the sky

black and white photograph of an old bathroom with a sink toilet and tub

the of a bag sitting on top of a bed next to a bag

4 Running the CLIP Model

First we'll use the pretrained CLIP model to extract features from the texts and images separetely.

```
In [27]: device = "cuda" if torch.cuda.is_available() else "cpu"
         clip_model, clip_preprocess = clip.load("ViT-B/32", device=device)
```

In [28]:
```
# You can check the model layers by printing the model.
# CLIP's model code is available at https://github.com/openai/CLIP/tree/main/clip
print(clip_model)
```

```
CLIP(
  (visual): VisionTransformer(
    (conv1): Conv2d(3, 768, kernel_size=(32, 32), stride=(32, 32), bias=False)
    (ln_pre): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
    (transformer): Transformer(
      (resblocks): Sequential(
        (0): ResidualAttentionBlock(
          (attn): MultiheadAttention(
            (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias
          )
          (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
          (mlp): Sequential(
            (c_fc): Linear(in_features=768, out_features=3072, bias=True)
            (gelu): QuickGELU()
            (c_proj): Linear(in_features=3072, out_features=768, bias=True)
          )
          (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        )
        (1): ResidualAttentionBlock(
          (attn): MultiheadAttention(
            (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias
          )
          (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
          (mlp): Sequential(
            (c_fc): Linear(in_features=768, out_features=3072, bias=True)
            (gelu): QuickGELU()
            (c_proj): Linear(in_features=3072, out_features=768, bias=True)
          )
          (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        )
        (2): ResidualAttentionBlock(
          (attn): MultiheadAttention(
            (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias
          )
          (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
          (mlp): Sequential(
            (c_fc): Linear(in_features=768, out_features=3072, bias=True)
            (gelu): QuickGELU()
            (c_proj): Linear(in_features=3072, out_features=768, bias=True)
          )
          (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        )
        (3): ResidualAttentionBlock(
          (attn): MultiheadAttention(
```

```
    (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias
  )
  (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=768, out_features=3072, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=3072, out_features=768, bias=True)
  )
  (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
)
(4): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias
  )
  (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=768, out_features=3072, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=3072, out_features=768, bias=True)
  )
  (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
)
(5): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias
  )
  (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=768, out_features=3072, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=3072, out_features=768, bias=True)
  )
  (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
)
(6): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias
  )
  (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=768, out_features=3072, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=3072, out_features=768, bias=True)
  )
  (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
)
(7): ResidualAttentionBlock(
  (attn): MultiheadAttention(
```

```
    (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias
  )
  (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=768, out_features=3072, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=3072, out_features=768, bias=True)
  )
  (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
)
(8): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias
  )
  (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=768, out_features=3072, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=3072, out_features=768, bias=True)
  )
  (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
)
(9): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias
  )
  (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=768, out_features=3072, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=3072, out_features=768, bias=True)
  )
  (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
)
(10): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias
  )
  (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=768, out_features=3072, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=3072, out_features=768, bias=True)
  )
  (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
)
(11): ResidualAttentionBlock(
  (attn): MultiheadAttention(
```

```
          (out_proj): NonDynamicallyQuantizableLinear(in_features=768, out_features=768, bias
        )
        (ln_1): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
        (mlp): Sequential(
          (c_fc): Linear(in_features=768, out_features=3072, bias=True)
          (gelu): QuickGELU()
          (c_proj): Linear(in_features=3072, out_features=768, bias=True)
        )
        (ln_2): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
      )
    )
  )
  (ln_post): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
)
(transformer): Transformer(
  (resblocks): Sequential(
    (0): ResidualAttentionBlock(
      (attn): MultiheadAttention(
        (out_proj): NonDynamicallyQuantizableLinear(in_features=512, out_features=512, bias=
      )
      (ln_1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
      (mlp): Sequential(
        (c_fc): Linear(in_features=512, out_features=2048, bias=True)
        (gelu): QuickGELU()
        (c_proj): Linear(in_features=2048, out_features=512, bias=True)
      )
      (ln_2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
    )
    (1): ResidualAttentionBlock(
      (attn): MultiheadAttention(
        (out_proj): NonDynamicallyQuantizableLinear(in_features=512, out_features=512, bias=
      )
      (ln_1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
      (mlp): Sequential(
        (c_fc): Linear(in_features=512, out_features=2048, bias=True)
        (gelu): QuickGELU()
        (c_proj): Linear(in_features=2048, out_features=512, bias=True)
      )
      (ln_2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
    )
    (2): ResidualAttentionBlock(
      (attn): MultiheadAttention(
        (out_proj): NonDynamicallyQuantizableLinear(in_features=512, out_features=512, bias=
      )
      (ln_1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
      (mlp): Sequential(
        (c_fc): Linear(in_features=512, out_features=2048, bias=True)
        (gelu): QuickGELU()
```

```
    (c_proj): Linear(in_features=2048, out_features=512, bias=True)
  )
  (ln_2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
)
(3): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=512, out_features=512, bias=
  )
  (ln_1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=512, out_features=2048, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=2048, out_features=512, bias=True)
  )
  (ln_2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
)
(4): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=512, out_features=512, bias=
  )
  (ln_1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=512, out_features=2048, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=2048, out_features=512, bias=True)
  )
  (ln_2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
)
(5): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=512, out_features=512, bias=
  )
  (ln_1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=512, out_features=2048, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=2048, out_features=512, bias=True)
  )
  (ln_2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
)
(6): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=512, out_features=512, bias=
  )
  (ln_1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=512, out_features=2048, bias=True)
    (gelu): QuickGELU()
```

```
    (c_proj): Linear(in_features=2048, out_features=512, bias=True)
  )
  (ln_2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
)
(7): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=512, out_features=512, bias=
  )
  (ln_1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=512, out_features=2048, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=2048, out_features=512, bias=True)
  )
  (ln_2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
)
(8): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=512, out_features=512, bias=
  )
  (ln_1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=512, out_features=2048, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=2048, out_features=512, bias=True)
  )
  (ln_2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
)
(9): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=512, out_features=512, bias=
  )
  (ln_1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=512, out_features=2048, bias=True)
    (gelu): QuickGELU()
    (c_proj): Linear(in_features=2048, out_features=512, bias=True)
  )
  (ln_2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
)
(10): ResidualAttentionBlock(
  (attn): MultiheadAttention(
    (out_proj): NonDynamicallyQuantizableLinear(in_features=512, out_features=512, bias=
  )
  (ln_1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
  (mlp): Sequential(
    (c_fc): Linear(in_features=512, out_features=2048, bias=True)
    (gelu): QuickGELU()
```

```
          (c_proj): Linear(in_features=2048, out_features=512, bias=True)
        )
        (ln_2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
      )
      (11): ResidualAttentionBlock(
        (attn): MultiheadAttention(
          (out_proj): NonDynamicallyQuantizableLinear(in_features=512, out_features=512, bias=
        )
        (ln_1): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
        (mlp): Sequential(
          (c_fc): Linear(in_features=512, out_features=2048, bias=True)
          (gelu): QuickGELU()
          (c_proj): Linear(in_features=2048, out_features=512, bias=True)
        )
        (ln_2): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
      )
    )
  )
  (token_embedding): Embedding(49408, 512)
  (ln_final): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
)
```

In [29]:
```
# First, we encode the captions into vectors in the shared embedding space.
# Since we're using a Transformer as the text encoder, we need to tokenize the text f
text_tokens = clip.tokenize(first_captions).to(device)
with torch.no_grad():
    text_features = clip_model.encode_text(text_tokens)

# Sanity check, print the shape
print(text_features.shape)

# Note: For your implementations, use the above clip_model.encode_text() function. Av
```

```
torch.Size([10, 512])
```

In [30]:
```
# Then, we encode the images into the same embedding space.
processed_images = [
    clip_preprocess(Image.fromarray(img)).unsqueeze(0)
    for img in first_images
]
images_tensor = torch.cat(processed_images, dim=0).to(device)

with torch.no_grad():
    image_features = clip_model.encode_image(images_tensor)

# sanity check, print the shape
```

```
        print(image_features.shape)

        # Note: For your implementations, use the above clip_model.encode_image() function. A
```

```
torch.Size([10, 512])
```

Open `cs231n/clip_dino.py` and implement `get_similarity_no_loop` to compute similarity scores between text features and image features. Test your implementation below, you should see relative errors less than 1e-5.

```
In [31]: from cs231n.clip_dino import get_similarity_no_loop
         torch.manual_seed(231)
         np.random.seed(231)
         M, N, D = 5, 6, 10

         test_text_features = torch.randn(N, D)
         test_image_features = torch.randn(M, D)
         out = get_similarity_no_loop(test_text_features, test_image_features)

         expected_out = np.array([
             [ 0.1867811 , -0.23494351,  0.44155994, -0.18950461,  0.00100103],
             [ 0.17905031, -0.25469488, -0.64330417,  0.25097957, -0.09327742],
             [-0.4407011 , -0.4365381 ,  0.32857686, -0.3765278 ,  0.01049389],
             [ 0.24815483,  0.42157224, -0.08459304,  0.14132318, -0.26935193],
             [ 0.02309848, -0.01441101,  0.5469337 ,  0.6018773 ,  0.21581158],
             [ 0.41579214, -0.014449  , -0.7242257 ,  0.39348006,  0.0822239 ],
         ]).astype(np.float32)

         print("relative error: ", rel_error(out.numpy(), expected_out))
```

```
relative error:  1.3374006e-06
```

```
In [34]: # Let's visualize the similarities between our batch of images and their captions.

         similarities = get_similarity_no_loop(text_features, image_features).cpu().detach().f

         plt.figure(figsize=(20, 14))
         plt.imshow(similarities.astype(np.float32), vmin=0.1, vmax=0.3)
         plt.yticks(range(len(text_features)), first_captions, fontsize=18)
         plt.xticks([])
         for i, image in enumerate(first_images):
             plt.imshow(to_uint8_rgb(image), extent=(i - 0.5, i + 0.5, -1.6, -0.6), origin="lo
         for x in range(similarities.shape[1]):
             for y in range(similarities.shape[0]):
                 plt.text(x, y, f"{similarities[y, x]:.2f}", ha="center", va="center", size=12

         for side in ["left", "top", "right", "bottom"]:
```

```
    plt.gca().spines[side].set_visible(False)

plt.xlim([-0.5, len(image_features) - 0.5])
plt.ylim([len(text_features) + 0.5, -2])

plt.title("Cosine similarity between text and image features", size=20)
plt.show()
```

5 Zero Shot Classifier

You will be able to see a high similarity between matching image-caption pairs above. We can leverage this property to design an image classifier that doesn't require any labeled data (i.e., a zero-shot classifier). Each class can be represented using an appropriate natural language description, and any input image will be classified into the class whose description has the highest similarity with the image in CLIP's embedding space.

Implement `clip_zero_shot_classifier` in `cs231n/clip_dino.py` and test it below. You should be able to see the following predictions:

['a person', 'an animal', 'an animal', 'food', 'a person', 'a landscape', 'other', 'other', 'other', 'a person']

```
In [35]: from cs231n.clip_dino import clip_zero_shot_classifier

         classes = ["a person", "an animal", "food", "a landscape", "other"]
```

```
        pred_classes = clip_zero_shot_classifier(
            clip_model, clip_preprocess, first_images, classes, device)

        print(pred_classes)
```

```
['a person', 'an animal', 'an animal', 'food', 'a person', 'a landscape', 'other', 'other', 'ot
```

Run the cell below to visualize the predictions. As you can see, CLIP offers a straightforward way to perform reasonable zero-shot classification across any class taxonomy.

CLIP was the first model to outperform standard supervised training on ImageNet classification without using any ImageNet images or labels (The original CLIP paper has many such interesting experiments and analysis).

In [36]:
```
# Visualize the zero shot predictions
for i, (pred_class, image) in enumerate(zip(pred_classes, first_images)):
    plt.imshow(to_uint8_rgb(image))
    plt.axis('off')
    plt.title(pred_class)
    plt.show()
```

an animal

an animal

food

a person

a landscape

other

other

other

6 Image Retrieval using CLIP

Just as we used CLIP to retrieve the matching class name for each image, we can also use it to retrieve matching images from text inputs (semantic image retrieval). Implement the `CLIPImageRetriever` in `cs231n/clip_dino.py` and test it by running the two cells below. The expected top 2 outputs for each query are provided in the comments.

```
In [37]: from cs231n.clip_dino import CLIPImageRetriever
         clip_retriever = CLIPImageRetriever(clip_model, clip_preprocess, first_images, device)

In [38]: query = "sports"  # tennis, skateboard
         # query = "black and white"  # bathroom, zerbas
         img_indices = clip_retriever.retrieve(query)

         for img_index in img_indices:
             plt.imshow(to_uint8_rgb(first_images[img_index]))
             plt.axis('off')
             plt.show()
```

Inline Question 2 -

CLIP learns to align image and text representations in a shared latent space using a contrastive loss. How would you extend this idea to more than two modalities?

Your Answer : To extend CLIP to more than two modalities, we can give each modality its own encoder, such as an image encoder, text encoder, audio encoder, video encoder, or sensor encoder, and project all outputs into a shared embedding space with the same feature dimension. For matched samples, such as an image, caption, and audio clip describing the same event,

their embeddings should be pulled close together, while embeddings from unmatched samples in the batch should be pushed apart. To train this is to use pairwise contrastive losses between all modality pairs, for example image-text, image-audio, and text-audio, then sum or average these losses.

7 DINO

As mentioned earlier, models trained with vanilla contrastive learning methods such as SimCLR and CLIP require very large batch sizes. This makes them computationally expensive and limits their accessibility. Subsequent works, like BYOL, propose an alternative approach that avoids the need for numerous negative samples by using a student-teacher framework. This method performs surprisingly well and was later adopted by DINO .

Similar to SimCLR, DINO is trained to maximize the agreement between two vectors derived from different views of the same image. However, unlike SimCLR, DINO uses two separate encoders which are trained differently. The student network is updated via backpropagation to match the outputs of the teacher network. The teacher network is not updated via backpropagation; instead, its weights are updated using an exponential moving average (EMA) of the student's weights. This means that the teacher model evolves more slowly and provides a stable target for the student to learn from.

Run the cell below to visualize the DINO training pipeline.

```
In [39]: from IPython.display import Image as DisplayImage
         DisplayImage(os.path.join(ASSIGNMENT_DIR, 'dino.gif'))
```

```
Out[39]: <IPython.core.display.Image object>
```

```
In [40]: # first let's get rid of the CLIP model that's currently using memory
         del clip_model
         # Uncomment the following if you are using GPU runtime
         # torch.cuda.empty_cache()
         # torch.cuda.ipc_collect()
```

```
In [41]: # Load smallest dino model. ViT-S/8. Here ViT-S has ~22M parameters and
         # works on 8x8 patches.
         dino_model = torch.hub.load('facebookresearch/dino:main', 'dino_vits8')
         dino_model.eval().to(device)
```

```
Downloading: "https://github.com/facebookresearch/dino/zipball/main" to /data/users/huanghao/.
Downloading: "https://dl.fbaipublicfiles.com/dino/dino_deitsmall8_pretrain/dino_deitsmall8_pret
100%|| 82.7M/82.7M [00:09<00:00, 8.76MB/s]
```

```
Out[41]: VisionTransformer(
           (patch_embed): PatchEmbed(
             (proj): Conv2d(3, 384, kernel_size=(8, 8), stride=(8, 8))
           )
           (pos_drop): Dropout(p=0.0, inplace=False)
           (blocks): ModuleList(
```

```
    (0-11): 12 x Block(
      (norm1): LayerNorm((384,), eps=1e-06, elementwise_affine=True)
      (attn): Attention(
        (qkv): Linear(in_features=384, out_features=1152, bias=True)
        (attn_drop): Dropout(p=0.0, inplace=False)
        (proj): Linear(in_features=384, out_features=384, bias=True)
        (proj_drop): Dropout(p=0.0, inplace=False)
      )
      (drop_path): Identity()
      (norm2): LayerNorm((384,), eps=1e-06, elementwise_affine=True)
      (mlp): Mlp(
        (fc1): Linear(in_features=384, out_features=1536, bias=True)
        (act): GELU(approximate='none')
        (fc2): Linear(in_features=1536, out_features=384, bias=True)
        (drop): Dropout(p=0.0, inplace=False)
      )
    )
  )
  (norm): LayerNorm((384,), eps=1e-06, elementwise_affine=True)
  (head): Identity()
)
```

In [42]:
```
# the image we will be playing around with
sample_image = Image.fromarray(first_images[0]).convert("RGB")
sample_image
```

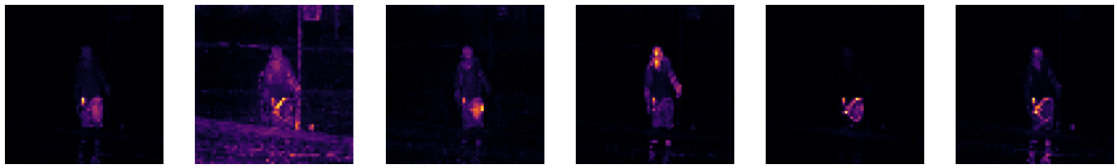
Out[42]:

8 DINO Attention Maps

Since the loaded DINO checkpoint is based on the ViT architecture, we can visualize what each attention head is focusing on. The code below generates heatmaps showing which patches of the original image the [CLS] token attends to across the various heads in the final layer. Although this model was trained using a self-supervised objective without any explicit instruction to recognize "structure" in images, still...

Do you notice any patterns?

In [43]:
```
# Preprocess
from torchvision import transforms as T
transform = T.Compose([
    T.Resize((480, 480)),
    T.ToTensor(),
    T.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225)),
])
img_tensor = transform(sample_image)
w, h = img_tensor.shape[1:]
img_tensor = img_tensor[None].to(device)

# Extract attention
with torch.no_grad():
    attn = dino_model.get_last_selfattention(img_tensor)[0, :, 0, 1:]
nh, tokens = attn.shape
w_feat, h_feat = w // 8, h // 8
attn = attn.reshape(nh, w_feat, h_feat)
attn = torch.nn.functional.interpolate(attn.unsqueeze(0), scale_factor=8, mode="neares

# Plot attention heads
fig, axes = plt.subplots(1, nh, figsize=(3 * nh, 3))
for i in range(nh):
    ax = axes[i] if nh > 1 else axes
    ax.imshow(attn[i], cmap='inferno')
    ax.axis('off')
plt.show()
```



```
In [44]: # Extract patch token features and discard [CLS] token.
         with torch.no_grad():
             all_tokens = dino_model.get_intermediate_layers(img_tensor, n=1)[0]  # (1, 1+N, D
             patch_tokens = all_tokens[:, 1:, :]  # (N, D)

         print(img_tensor.shape)
         print(all_tokens.shape)
         print(patch_tokens.shape)
```

```
torch.Size([1, 3, 480, 480])
torch.Size([1, 3601, 384])
torch.Size([1, 3600, 384])
```

Inline Question 3

How do we get the tensor shapes printed above? Explain your answer.

YourAnswer :

The image is then divided into non-overlapping patches. Since the ViT uses patch size 8×8, $\frac{480}{8} = 60$

so along height there are 60 patches, and along width there are also 60 patches. Therefore the total number of image patches is

$60 \times 60 = 3600$.

Each patch is embedded into a 384-dimensional vector, because this ViT model has embedding dimension

$D = 384$.

So the patch token tensor has shape

`torch.Size([1, 3600, 384])`.

The second tensor shape

`torch.Size([1, 3601, 384])`

includes one extra $[CLS]$ token. ViT prepends a learnable class token to the patch tokens, so the total number of tokens becomes

$3600 + 1 = 3601$.

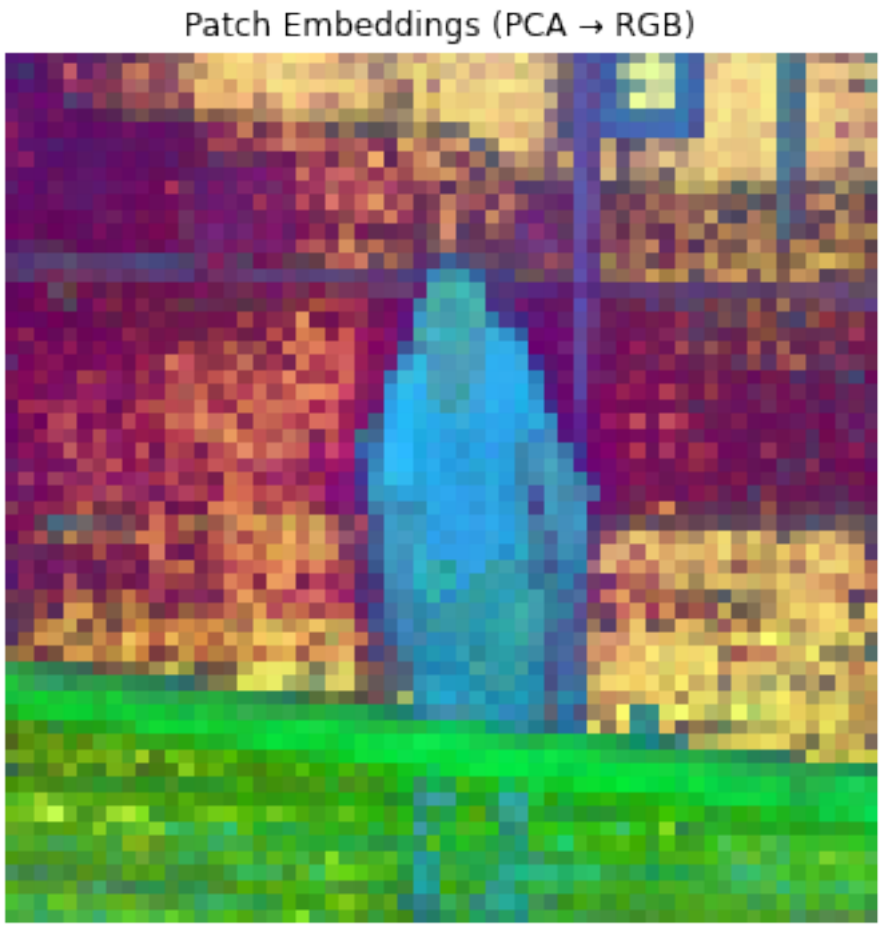
9 DINO Features

To understand what the model is encoding in each patch, we can visualize the contents of each patch token. Since these embeddings are high-dimensional and difficult to interpret directly, we'll use PCA to identify the directions of highest variance in the feature space.

In the next cell, we visualize the three principal directions of variance in the feature space. This reveals the dominant structure that the patch embeddings are capturing.

```
In [46]: from sklearn.decomposition import PCA

         np.random.seed(231)

         # PCA
         pca = PCA(n_components=3)
         patch_pca = pca.fit_transform(patch_tokens.cpu().numpy()[0])
```

```
# Normalize PCA components to [0, 1] for RGB display
patch_rgb = (patch_pca - patch_pca.min(0)) / (patch_pca.max(0) - patch_pca.min(0))

# Reshape to image grid (60x60, 3)
patch_rgb_img = patch_rgb.reshape(60, 60, 3)

# Show as image
plt.figure(figsize=(6, 6))
plt.imshow(patch_rgb_img)
plt.axis('off')
plt.title("Patch Embeddings (PCA  RGB)")
plt.show()
```



Inline Question 4 -

What kind of structure do you see in the visualization above? What does it imply when a region consistently appears in a specific color? What does it mean when two regions have distinctly different color? Remember that PCA reveals the directions of highest variance in the feature space across all patches. A patch's color reflects its distinct feature content.

Your Answer :

The visualization shows that the patch features are not random: they form coherent spatial regions. The central human-like object appears mostly in blue, the grass at the bottom appears green, and the background has different reddish/yellow/purple regions. This suggests that the model's patch embeddings have learned meaningful visual structure, such as object foreground, background, texture, and scene layout.

When a region consistently appears in a specific color, it means the patches in that region have similar feature representations after PCA projection. In other words, the model considers those patches to contain similar visual or semantic information.

10 A Simple Segmentation Model over DINO Features

In the previous section, we saw that DINO features can provide surprisingly good segmentation cues. Now, let's put that idea to the test by training a simple segmentation model on the DAVIS dataset. The DAVIS dataset (Densely Annotated VIdeo Segmentation) was created for video object segmentation tasks. It provides frame-by-frame, pixel-level annotations of objects within videos. For this experiment, we'll train our model using the annotations from just a single frame of a video and see how well it performs on the remaining frames of the same.

Our model will be intentionally minimal: we'll extract DINO features per patch and train a lightweight per-patch classifier using only the patches from that one annotated frame. Typically, you would train on the full dataset and evaluate on a separate validation set containing different videos. But here, we will test the one-shot capabilities of DINO features.

In [47]:
```
from cs231n.clip_dino import DavisDataset

# A helper class to work with DAVIS dataset.
# It may take ~5 minutes on the first run of this cell to download the dataset.
davis_ds = DavisDataset()

# Get a specific test video. Do NOT change this for submission.
frames, masks = davis_ds.get_sample(7)
num_classes = masks.max() + 1

print(frames.shape, masks.shape, num_classes)
```

```
Downloading and preparing dataset Unknown size (download: Unknown size, generated: Unknown size


Dl Completed...: 0 url [00:00, ? url/s]


Dl Size...: 0 MiB [00:00, ? MiB/s]


Extraction completed...: 0 file [00:00, ? file/s]


Generating splits...:   0%|          | 0/2 [00:00<?, ? splits/s]
```

```
Generating train examples...: 0 examples [00:00, ? examples/s]

Shuffling /data/users/huanghao/tensorflow_datasets/davis/480p/2.1.0.incompleteQDVU2G/davis-tra

Generating validation examples...: 0 examples [00:00, ? examples/s]

Shuffling /data/users/huanghao/tensorflow_datasets/davis/480p/2.1.0.incompleteQDVU2G/davis-val

WARNING:absl:`FeatureConnector.dtype` is deprecated. Please change your code to use NumPy with
WARNING:absl:`FeatureConnector.dtype` is deprecated. Please change your code to use NumPy with

Dataset davis downloaded and prepared to /data/users/huanghao/tensorflow_datasets/davis/480p/2
video soapbox  99 frames
(99, 480, 854, 3) (99, 480, 854, 1) 4
```

```
In [48]: # Get DINO patch features and corresponding class labels for a middle frame
         train_fi = 40
         X_train = davis_ds.process_frames(frames[train_fi:train_fi+1], dino_model, device)[0]
         Y_train = davis_ds.process_masks(masks[train_fi:train_fi+1], device)[0]
         print(X_train.shape, Y_train.shape)
```

```
torch.Size([3600, 384]) torch.Size([3600])
```

Complete the implementation of the `DINOSegmentation` class in `cs231n/clip_dino.py`, and test it by running the two cells below. You should achieve a mean IoU greater than 0.45 on the first test frame and greater than 0.50 on the last test frame. To prevent overfitting on the training patch features, consider designing a very lightweight model (e.g., a linear layer or a 2-layer MLP) and applying appropriate weight decay.

You may use GPU runtime to speed up training and evaluation. Make sure to rerun the entire notebook if you change runtime type.

```
In [49]: from cs231n.clip_dino import DINOSegmentation, compute_iou
         torch.manual_seed(231)
         np.random.seed(231)
         dino_segmentation = DINOSegmentation(device, num_classes)
         dino_segmentation.train(X_train, Y_train, num_iters=500)


         # Test on first, middle, and last frame
         ious = []
         test_fis = [0, train_fi, 98]
         gt_viz = []
```

```
        pred_viz = []
        for fi in test_fis:
          X_test = davis_ds.process_frames(frames[fi:fi+1], dino_model, device)[0]
          Y_test = davis_ds.process_masks(masks[fi:fi+1], device)[0]
          Y_pred = dino_segmentation.inference(X_test)
          iou = compute_iou(Y_pred, Y_test, num_classes)
          ious.append(iou)

          gt_viz.append(davis_ds.mask_frame_overlay(Y_test, frames[fi]))
          pred_viz.append(davis_ds.mask_frame_overlay(Y_pred, frames[fi]))

        gt_viz = np.concatenate(gt_viz, 1)
        pred_viz = np.concatenate(pred_viz, 1)
```

```
In [50]: print(f"Mean IoU on first test frames: {ious[0]:.3f}")  # should be >0.45
         print(f"Mean IoU on last test frames: {ious[2]:.3f}")  # should be >0.50
```

```
Mean IoU on first test frames: 0.449
Mean IoU on last test frames: 0.543
```

Now let's visualize the results. Run the two cells below to display the ground truth and predicted segmentation masks for the first, middle, and last frames. Note that the middle frame is part of the training set, while the other frames are unseen.

```
In [51]: Image.fromarray(gt_viz)
```

Out[51]:

```
In [52]: Image.fromarray(pred_viz)
```

Out[52]:

Now run the following three cells to evaluate and visualize the entire video. You should achieve a mean IoU greater than 0.55. The saved visualization video may take some time to write on the server, but you can open it locally after it finishes.

```
In [53]: # Run on all frames
         ious = []
         gt_viz = []
         pred_viz = []
         for fi in range(len(frames)):
           if fi % 20 == 0:
             print(f"{fi} / {len(frames)}")
           X_test = davis_ds.process_frames(frames[fi:fi+1], dino_model, device)[0]
           Y_test = davis_ds.process_masks(masks[fi:fi+1], device)[0]
           Y_pred = dino_segmentation.inference(X_test)
           iou = compute_iou(Y_pred, Y_test, num_classes)
           ious.append(iou)

           gt_viz.append(davis_ds.mask_frame_overlay(Y_test, frames[fi]))
           pred_viz.append(davis_ds.mask_frame_overlay(Y_pred, frames[fi]))

         gt_viz = np.stack(gt_viz)  # T x H x W x 3
         pred_viz = np.stack(pred_viz)  # T x H x W x 3
         final_viz = np.concatenate([gt_viz, pred_viz], -2)  # T x H x 2W x 3
```

```
0 / 99
20 / 99
40 / 99
60 / 99
80 / 99
```

```
In [54]: print(f"Mean IoU on all frames: {sum(ious) / len(ious):.3f}")  # should be >0.55
```

```
Mean IoU on all frames: 0.636
```

```
In [55]: def write_video_from_array(array, output_path, fps = 12):
             T, H, W, _ = array.shape
             fourcc = cv2.VideoWriter_fourcc(*'mp4v')
             out = cv2.VideoWriter(output_path, fourcc, fps, (W, H))
             for i in range(T):
                 frame = array[i]
                 out.write(frame)
             out.release()
             print(f"Video saved to {output_path}")
```

```
output_path = os.path.join(ASSIGNMENT_DIR, "dino_res.mp4")
write_video_from_array(final_viz, output_path)
```

```
Video saved to /data/users/huanghao/a3/assignment3/dino_res.mp4
```

Inline Question 5 -

If you train a segmentation model on CLIP ViT's patch features, do you expect it to perform better or worse than DINO? Why should that be the case?

Your Answer :

I would expect segmentation using CLIP ViT patch features to perform worse than DINO patch features in this setting. CLIP is trained with an image-text contrastive objective, so its strongest supervision is at the global image-text alignment level: the model learns features useful for matching an entire image to a caption, but it is not directly encouraged to make every local patch correspond to a coherent object region.

DINO, on the other hand, is trained with self-supervised image-level consistency across multiple crops, and in ViTs this tends to produce patch features and attention maps that naturally separate foreground objects from background. Therefore DINO patch embeddings often contain stronger local spatial and object-level structure, making them better suited for training a lightweight patch-wise segmentation classifier.

In [ ]: